به نام خداوند جان و خرد

گزارش جامع آزمایش‌ها

طبقه‌بندی تصاویر وسایل نقلیه ترافیکی

از شبکه CNN ساده تا ResNet۱۸ و تشخیص نمونه‌های خارج از توزیع (OOD)

نام و نام خانوادگی: ستایش هجاوری

مهر ۱۴۰۵

۱. مقدمه و بیان مسئله

۱.۱ تعریف مسئله

هدف این پروژه ساخت یک سامانه بینایی ماشین است که تصویر یک وسیله نقلیه ترافیکی را دریافت کند و آن را در یکی از **۸ کلاس** تعریف‌شده دسته‌بندی کند. از نظر ریاضی، مسئله یک **طبقه‌بندی تک‌برچسبی چندکلاسه** است: به‌ازای هر تصویر x باید تابعی مانند f_θ یاد گرفته شود که برچسب $y \in \{۱, \dots, ۸\}$ را پیش‌بینی کند و هر تصویر دقیقاً به یک کلاس تعلق دارد.

$$\hat{y}(x) = \arg\max_{k \in \{۱,\dots,۸\}} f_\theta(x)_k$$

کلاس‌ها بر اساس نام‌های رایج در مجموعه‌داده در جدول زیر آمده‌اند.

جدول ۱ — کلاس‌های مسئله و ویژگی‌های آن‌ها

نام کلاس	معادل فارسی	ویژگی ظاهری و دشواری احتمالی
ambulance	آمبولانس	رنگ و علائم مشخص؛ در شبکه‌های ساده گاهی با وانت/کامیونت اشتباه می‌شود
autobus	اتوبوس	بدنه ی بزرگ و پنجره‌های ردیفی؛ معمولاً ساده‌ترین کلاس
kamyun	کامیون	بدنه ی بزرگ و باری؛ بیشترین اشتباه با kamyunet
kamyunet	کامیونت	کامیون سبک؛ از نظر اندازه و شکل شبیه kamyun
minibus	مینی‌بوس	حدواسط اتوبوس و ون؛ گاهی با kamyun/kamyunet اشتباه می‌شود
savari	خودروی سواری	کلاس پرتعداد؛ تنوع زیاد مدل و رنگ
taxi	تاکسی	رنگ و تابلوی مشخص؛ معمولاً دقت بالا
vanet	وانت	کمترین نمونه (۶٪)؛ بیشترین اشتباه با savari

چند چالش اصلی در این مسئله وجود دارد که مسیر آزمایش‌ها را شکل داده‌اند:

- **شباهت بین‌کلاسی:** kamyun و kamyunet و نیز vanet و savari از نظر ظاهری بسیار نزدیک‌اند.
- **نامتوازن بودن کلاس‌ها:** کلاس vanet تنها ۶٪ داده ی آموزشی را تشکیل می‌دهد (نسبت بیشترین به کمترین کلاس ≈ ۲.۴۴).
- **تنوع نسبت ابعاد تصاویر:** تصاویر عمدتاً عمودی (portrait) هستند (میانگین نسبت w/h برابر ۰.۸۱۲)؛ بنابراین تبدیل به ورودی مربعی باید با دقت انجام شود (بحث Letterbox در بخش ۲.۲).

- **کم بودن داده نسبت به ظرفیت مدل:** تنها ۳,۳۹۲ تصویر برای آموزش مدل‌هایی با ده‌ها میلیون پارامتر وجود دارد؛ نتیجه، **بیش‌برازش** است.
- **مسئله ی دنیای باز (Open-set)**: در کاربرد واقعی ممکن است وسیله‌ای ظاهر شود که جزو ۸ کلاس نیست (در این پروژه: **Neysan/نیسان**). یک طبقه‌بند معمولی ناچار است همچنان یکی از ۸ کلاس را انتخاب کند.

۱.۲ رویکرد کلی: آزمایش‌های کنترل‌شده (Ablation)

برای اینکه اثر هر تصمیم طراحی به‌روشنی قابل‌سنجش باشد، آزمایش‌ها به‌صورت **کنترل‌شده** انجام شده‌اند: در هر آزمایش فقط یک عامل نسبت به پیکربندی مرجع تغییر می‌کند (مثلاً فقط Dropout یا فقط نوع Pooling) و بقیه ی عوامل—داده، تقسیم آموزش/اعتبارسنجی، seed، بهینه‌ساز و تعداد اپوک‌ها—ثابت می‌ماند. مجموعه ی آزمون (Test) در طول مقایسه ی مدل‌ها دست‌نخورده نگه داشته شده و انتخاب مدل فقط بر پایه ی مجموعه ی اعتبارسنجی انجام شده است. این روش‌شناسی مستقیماً از الزامات پروژه است و در بخش ۲.۸ بیشتر شرح داده می‌شود.

۱.۳ نقشه ی راه و تکامل مدل

مسیر تکامل مدل را می‌توان در چهار فاز خلاصه کرد. جدول زیر خلاصه ی این مسیر است و در ادامه هر فاز با جزئیات کامل بررسی می‌شود.

جدول ۲ — خلاصه ی فازهای آزمایش

فاز	آزمایش‌ها	پرسش اصلی	نتیجه ی کلیدی
۰	ممیزی داده	آیا داده تمیز و بدون نشت است؟	۲۳ تصویر تکراری حذف شد؛ ۳,۳۹۲ تصویر نهایی
۱	۰۱ تا ۰۹	یک CNN ساده از صفر چقدر می‌تواند یاد بگیرد و کدام ترفند آن را بهتر می‌کند؟	سقف ≈ ۷۸–۸۰٪؛ بیش‌برازش شدید؛ GAP بدتر (۵۸.۴۷٪)
۲	۱۰ و ۱۱	آیا یادگیری انتقالی (ResNet۱۸) کمک می‌کند؟ کدام افزایش داده بهتر است؟	۹۶.۰۲٪ با Full Fine-Tuning؛ تا ۹۸.۲۳٪ با افزایش داده
۳	۱۲ و ۱۳	آیا Letterbox و Dropout در مدل نهایی سودمندند؟	Non-Letterbox و بدون ۹۸.۳۸٪: Dropout (اعتبارسنجی)، ۹۸.۵٪ (آزمون)
۴	۱۴	آیا کلاس ناشناخته ی Neysan قابل تشخیص است؟	Mahalanobis ۲۸.۱۶٪، Energy ۵.۹۷٪، ترکیبی ۳۱.۵۷٪؛ ۴۰۱ مورد جاافتاده

۲. داده‌ها و تنظیمات مشترک آزمایش‌ها

این فصل همه ی مواردی را که در تمام آزمایش‌ها مشترک است—به‌همراه مبانی ریاضی و مفهومی لازم—شرح می‌دهد تا در فصل‌های بعد فقط به «تفاوت‌ها» پرداخته شود.

۲.۱ ممیزی داده (Data Audit)

پیش از هر آزمایشی، نسخه ی فعلی مجموعه‌داده با یک pipeline پیش‌پردازش بازنویسی‌شده (`src/preprocess.py`) و نوت‌بوک `00_data_audit.ipynb` بررسی شد. مجموعه‌داده از سه پوشه تشکیل شده است:

جدول ۳ — تعداد فایل‌ها در پوشه‌های مجموعه‌داده

پوشه	تعداد فایل
train	۱,۷۷۶
test	۴۰۰
unclean	۱,۶۳۹

بررسی تکراری‌ها با MD۵. برای هر فایل یک چکیده ی ۱۲۸ بیتی $h(x) = \mathrm{MD5}(x)$ محاسبه شد. دو فایل با چکیده ی برابر، با احتمالی بسیار نزدیک به یک، بایت‌به‌بایت یکسان‌اند.

یعنی برابری چکیده عملاً به معنای تکراری بودن است. توجه کنید که MD۵ فقط **تکراری‌های دقیق** را پیدا می‌کند و تصاویر تقریباً مشابه (مثلاً تغییر اندازه‌یافته یا با فشرده ی‌سازی متفاوت) را تشخیص نمی‌دهد؛ این یک محدودیت شناخته‌شده است.

نتایج بررسی: • تکراری داخلی در هر پوشه، • تضاد برچسب، • فایل ناخوانا، • تصویر کوچک‌تر از ۳۲×۳۲ و • تصویر با نسبت ابعاد پرت (آستانه‌های ۰.۲۵ > یا ۴ <). آمار نسبت ابعاد پوشه ی unclean (تعداد ۱,۶۳۹): میانگین ۰.۸۱۲، انحراف معیار ۰.۱۱۱، کمینه ۰.۳۸۲، میانه ۰.۷۸۴ و بیشینه ۱.۳۷۸؛ یعنی تصاویر عمدتاً عمودی هستند.

جدول ۴ — تصاویر تکراری بین مجموعه‌ها (نشت داده)

مقایسه	تعداد تطابق	کلاس‌های درگیر
unclean ↔ train	۱۵	ambulance (۸)، kamyunet (۴)، minibus (۲)، autobus (۱)
unclean ↔ test	۸	ambulance (۶)، minibus (۲)

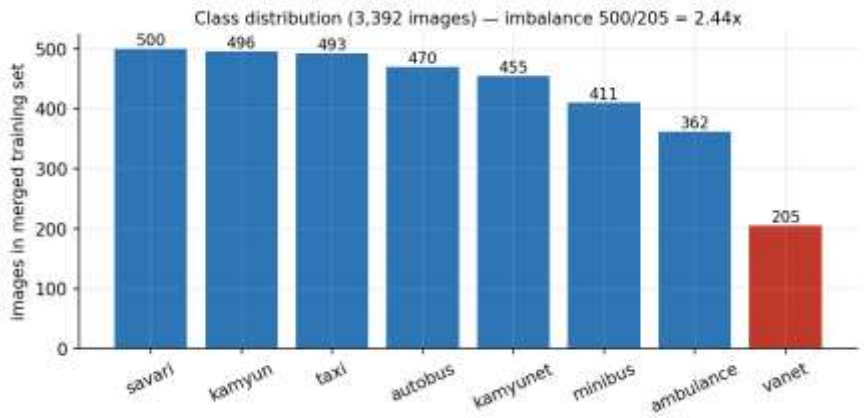
نکته مهم: تکراری بودن یک تصویر در مجموعه ی آموزش و آزمون یا اعتبارسنجی باعث **نشت داده** (**Data Leakage**) می‌شود: مدل تصویری را که قبلاً دیده است دوباره می‌بیند و دقت گزارش‌شده بیش از واقعیت خواهد بود. به همین دلیل ۲۳ تصویر تکراری (۱۵ مورد `duplicate_in_train` و ۸ مورد `duplicate_in_test`) از پوشه ی unclean حذف شد؛ ۱,۶۱۶ تصویر باقی ماند.

مجموعه ی آموزشی نهایی از ادغام train و unclean پاک‌سازی‌شده ساخته شد: ۳۳۹۲ = ۱۶۱۶ + ۱۷۷۶ تصویر.

توزیع کلاس‌ها:

جدول ۵ — توزیع کلاس‌ها در مجموعه ی ادغام‌شده (مجموعه ی آموزشی)

کلاس	تعداد	سهم
savari	۵۰۰	۱۴.۷٪
kamyun	۴۹۶	۱۴.۶٪
taxi	۴۹۳	۱۴.۵٪
autobus	۴۷۰	۱۳.۹٪
kamyunet	۴۵۵	۱۳.۴٪
minibus	۴۱۱	۱۲.۱٪
ambulance	۳۶۲	۱۰.۷٪
vanet	۲۰۵	۶.۰٪
Total	**۳,۳۹۲**	**۱۰۰٪**



شکل ۱ — توزیع کلاس‌ها؛ vanet با ۲۰۵ تصویر کمترین سهم (۶.۰٪) را دارد.

نسبت بیشترین به کمترین کلاس ۲.۴۴ ≈ ۵۰۰/۲۰۵ است؛ یعنی ناترازی **متوسط** (نه شدید). این ناترازی در آزمایش ۰۷ بررسی می‌شود. پس از تقسیم ۲۰/۸۰، مجموعه ی آموزش ۲,۷۱۳ و مجموعه ی اعتبارسنجی ۶۷۹ تصویر دارد و همین تقسیم در تمام آزمایش‌ها ثابت است.

۲.۲ پیش‌پردازش Letterbox: مفهوم و ریاضیات

الف) چرا به تغییر اندازه نیاز داریم؟

شبکه‌ای که لایه ی تماماً متصل (FC) دارد—مانند SimpleCNN با ورودی `Linear(200704→128)`—فقط ورودی با اندازه ی ثابت می‌پذیرد. اما تصاویر خام ابعاد متفاوتی دارند. سه راه معمول عبارت‌اند از:

(۱) تغییر اندازه ی مستقیم به ۲۲۴ × ۲۲۴ (کشیدگی/Distortion)

(۲) برش (از بین رفتن بخشی از خودرو)

(۳) Letterbox: تغییر اندازه با حفظ نسبت ابعاد و پر کردن حاشیه‌های خالی. در این پروژه Letterbox انتخاب اولیه بود.

ب) تعریف ریاضی

فرض کنید تصویر اصلی ابعاد $w \times h$ (عرض × ارتفاع) و اندازه ی هدف $S = ۲۲۴$ باشد. ضریب مقیاس **یکنواخت** (یکسان در دو محور) چنین تعریف می‌شود:

$$s = \min\left(\frac{S}{w},\ \frac{S}{h}\right), \qquad w' = \text{round}(s\,w), \quad h' = \text{round}(s\,h)$$

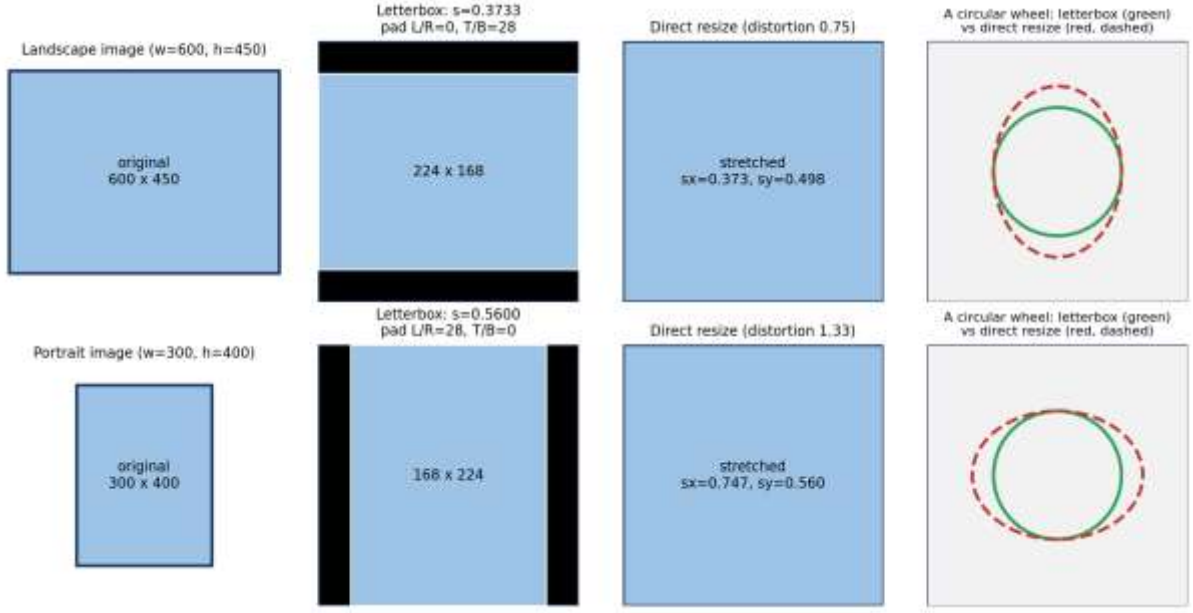
بُعد غالب دقیقاً برابر S می‌شود و بُعد دیگر کوچک‌تر یا مساوی S خواهد بود. فضای باقی‌مانده با رنگ ثابت (در اینجا **سیاه**) پر می‌شود و تصویر در مرکز قاب قرار می‌گیرد:

$$p_x = \left\lfloor \frac{S - w'}{۲} \right\rfloor, \qquad p_y = \left\lfloor \frac{S - h'}{۲} \right\rfloor$$

$$I_{\text{out}}(u, v) = \begin{cases} I_{\text{resized}}(u - p_x,\ v - p_y) & p_x \le u < p_x + w',\ p_y \le v < p_y + h' \\ \text{otherwise (black padding)} & \end{cases}$$

که در آن (u, v) مختصات پیکسل در قاب خروجی است. **مثال عددی:** تصویر افقی ۴۵۰ × ۶۰۰ با

$s = ۲۲۴/۶۰۰ = ۰.۳۷۳۳$ به ۱۶۸ × ۲۲۴ تبدیل و با ۲۸ پیکسل سیاه در بالا و پایین پر می‌شود. تصویر عمودی ۴۰۰ × ۳۰۰ با $s = ۲۲۴/۴۰۰ = ۰.۵۶$ به ۲۲۴ × ۱۶۸ تبدیل و با ۲۸ پیکسل سیاه در چپ و راست پر می‌شود (شکل بعد).



شکل ۲ — مقایسه ی Letterbox و تغییر اندازه ی مستقیم برای یک تصویر افقی (ردیف بالا) و یک تصویر عمودی (ردیف پایین). ستون آخر نشان می‌دهد یک چرخ دایره‌ای در تغییر اندازه ی مستقیم به بیضی تبدیل می‌شود.

یادآوری: در همه ی آزمایش‌های ۰۱ تا ۱۱ ورودی با Letterbox ساخته شده است؛ مقایسه ی مستقیم Letterbox و تغییر اندازه ی مستقیم تنها در آزمایش‌های ۱۲ و ۱۳ و روی ResNet۱۸ انجام شده است.

۲.۳ نرمال‌سازی ImageNet

پس از تبدیل به تنسور (مقادیر در $[۰,۱]$)، هر کانال با میانگین و انحراف معیار ImageNet استاندارد می‌شود:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c}, \quad \mu = (۰.۴۸۵, ۰.۴۵۶, ۰.۴۰۶), \quad \sigma = (۰.۲۲۹, ۰.۲۲۴, ۰.۲۲۵)$$

مزیت نرمال‌سازی این است که ورودی تقریباً میانگین صفر و واریانس واحد دارد؛ این امر شرایط بهینه‌سازی (Conditioning) را بهبود می‌دهد و برای مدل‌های پیش‌آموزش‌دیده روی ImageNet ضروری و سازگار با آمار آموزش آن‌هاست. در آزمایش ۱۰ اثر حذف آن نیز سنجیده می‌شود. (در یادداشت نوت‌بوک آمده که نسخه ی قبلیِ آزمایش ۰۱ نرمال‌سازی نداشت و نسخه ی فعلی دارد.)

۲.۴ تقسیم داده و تکرارپذیری

تقسیم آموزش/اعتبارسنجی با تابع

```
create_train_val_datasets(..., test_size=0.2, seed=42)
```

و به‌صورت **طبقه‌بندی‌شده** (**Stratified**) انجام می‌شود؛ یعنی نسبت کلاس‌ها در هر دو بخش تقریباً یکسان است:

نتیجه ۲,۷۱۳ تصویر آموزش و ۶۷۹ تصویر اعتبارسنجی است:

جدول ۶ — تعداد نمونه‌های اعتبارسنجی به‌تفکیک کلاس (از ستون Support در گزارش‌های طبقه‌بندی)

کلاس	ambulance	autobus	kamyun	kamyunet	minibus	savari	taxi	vanet	جمع
تعداد در اعتبارسنجی	۷۳	۹۴	۹۹	۹۱	۸۲	۱۰۰	۹۹	۴۱	۶۷۹

seed برابر ۴۲ است و با `set_seed()` در ابتدای هر نوت‌بوک ثابت می‌شود. شاهد روشن تکرارپذیری این است که شش اپوک اول آزمایش ۰۶ (زمان‌بند نرخ یادگیری) **دقیقاً** با شش اپوک اول آزمایش ۰۱ برابر است؛ زیرا تا اپوک ۶ نرخ یادگیری تغییری نکرده است.

۲.۵ معماری مرجع: SimpleCNN

معماری پایه که همه ی آزمایش‌های فاز ۱ از آن شروع می‌شود، یک CNN کوچک است:

```
Conv(3→32) → ReLU → Pool(2) → Conv(32→64) → ReLU → Pool(2)
Flatten → Linear(200704→128) → ReLU → Dropout → Linear(128→8) →
```

هر لایه ی کانولوشن روی ورودی x با هسته ی W عملیات زیر را انجام می‌دهد و خروجی ReLU برابر $\max(0, z)$ است:

$$y_k(i,j) = b_k + \sum_{c=1}^{C_{in}} \sum_{u,v} W_k(c,u,v)\, x_c(i+u, j+v), \qquad \text{ReLU}(z) = \max(0, z)$$

اندازه ی خروجی هر لایه از رابطه ی زیر به‌دست می‌آید که در آن n اندازه ی ورودی، k اندازه ی هسته، p حاشیه‌گذاری و s گام است:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

تعداد ویژگی‌های `Flatten` برابر $۶۴ \times ۵۶ \times ۵۶ = ۲۰۰,۷۰۴$ است؛ این عدد دقیقاً وقتی به‌دست می‌آید که کانولوشن‌ها اندازه را حفظ کنند (مثلاً conv ۳×۳ با $p = ۱$) و دو Pooling با گام ۲ اندازه را از ۲۲۴ به ۱۱۲ و سپس ۵۶ برسانند. بر این فرض، شمارش پارامترها چنین است:

جدول ۷ — ساختار و تعداد پارامترهای SimpleCNN (با فرض هسته ی ۳×۳)

لایه	اندازه ی خروجی	تعداد پارامتر	محاسبه
Conv۱ (۳→۳۲)	۲۲۴×۲۲۴×۳۲	۸۹۶	۳۲ × ۳۲۰۳۰۳
Pool ۱	۱۱۲×۱۱۲×۳۲	۰	—
Conv۲ (۳۲→۶۴)	۱۱۲×۱۱۲×۶۴	۱۸,۴۹۶	۶۴ × ۶۴۰۹۰۳۲
Pool ۲	۵۶×۵۶×۶۴	۰	—
FC۱ (۲۰۰۷۰۴→۱۲۸)	۱۲۸	۲۵,۶۹۰,۲۴۰	۱۲۸ × ۱۲۸۰۲۰۰۷۰۴
FC۲ (۱۲۸→۸)	۸	۱,۰۳۲	۸ × ۸۰۱۲۸
جمع		**۲۵,۷۱۰,۶۶۴**	

نتیجه کلیدی: حدود **۹۹.۹۲٪** پارامترهای شبکه در لایه ی FC۱ قرار دارد. این لایه هر موقعیت مکانی هر نقشه ی ویژگی را با یک وزن مستقل می‌بیند؛ پس ظرفیت حفظ کردن (Memorization) نمونه‌های آموزش بسیار بالاست. این مشاهده توضیح می‌دهد چرا آزمایش‌های بعدی (Dropout، GAP، weight decay) همگی روی همین بخش اثر می‌گذارند.

۲.۶ تابع هزینه و بهینه‌ساز

خروجی شبکه ۸ لوجیت $z \in \mathbb{R}^{۸}$ است که با Softmax به توزیع احتمال تبدیل می‌شود. تابع هزینه ی **CrossEntropy** برای نمونه ی با برچسب y برابر منفی لگاریتم احتمال کلاس درست است:

$$p_k = \frac{e^{z_k}}{\sum_{j=۱}^{۸} e^{z_j}}, \qquad \mathcal{L}_{CE} = -\log p_y$$

گرادیان نسبت به لوجیت‌ها فرم ساده‌ای دارد: $\partial \mathcal{L} / \partial z_k = p_k - \mathbb{۱}[k = y]$؛ یعنی هرچه پیش‌بینی از جواب اصلی دورتر باشد، گرادیان بزرگ‌تر است. بهینه‌ساز **Adam** با نرخ یادگیری $\eta = ۱۰^{-۳}$ از میانگین متحرک گرادیان و مربع آن استفاده می‌کند:

$$m_t = \beta_۱ m_{t-۱} + (۱ - \beta_۱) g_t, \quad v_t = \beta_۲ v_{t-۱} + (۱ - \beta_۲) g_t^{۲}, \quad \theta_{t+۱} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

با اندازه ی دسته ۳۲ و ۲,۷۱۳ نمونه، هر اپوک شامل ۸۵ = ⌈۲۷۱۳/۳۲⌉ گام بهینه‌سازی است؛ در ۱۰ اپوک در مجموع ۸۵۰ گام انجام می‌شود (این عدد در تحلیل weight decay به کار می‌آید).

۲.۷ معیارهای ارزیابی

اگر برای کلاس k تعداد مثبت‌های درست TP_k، مثبت‌های کاذب FP_k و منفی‌های کاذب FN_k باشد:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = \frac{2\,P_k R_k}{P_k + R_k}$$

$$\text{Accuracy} = \frac{\sum_k TP_k}{N}, \qquad \text{Macro-}F1 = \frac{1}{K}\sum_{k=1}^{K} F1_k$$

- **Macro-average** همه ی کلاس‌ها را با وزن برابر می‌شمارد و به عملکرد کلاس‌های کم‌نمونه (مثل vanet) حساس است؛ **Weighted** با Support وزن‌دهی می‌شود.
- **ماتریس درهم‌ریختگی (Confusion Matrix)**: درایه ی (i, j) تعداد نمونه‌های کلاس واقعی i است که کلاس j پیش‌بینی شده‌اند؛ قطر اصلی پیش‌بینی‌های درست است. در جدول‌های این گزارش، قطر با سبز و اشتباهات پررنگ با قرمز سایه‌دار شده‌اند.
- **شکاف آموزش/اعتبارسنجی (Generalization Gap)** برابر است با $\text{Acc}_{train} - \text{Acc}_{val}$ و شاخص اصلی بیش‌برازش است.
- **تفاوت دقت و خطا:** دقت فقط به argmax وابسته است، اما خطا به **اطمینان** پیش‌بینی هم وابسته است. یک پیش‌بینی غلط با اطمینان بالا هزینه ی $-\log p_y$ بزرگی دارد. به همین دلیل ممکن است خطای اعتبارسنجی افزایش یابد در حالی که دقت ثابت بماند یا بهتر شود.

۲.۸ پروتکل ارزیابی

- **مجموعه ی آزمون (Test)** در مقایسه ی آزمایش‌ها استفاده نشد؛ فقط مجموعه ی اعتبارسنجی برای مقایسه و انتخاب نقطه ی ذخیره (Checkpoint) به کار رفت.
- **Checkpointing:** در هر اجرا اپوکی با بیشترین دقت اعتبارسنجی ذخیره می‌شود و نتیجه در `results/experiment_results.json` با کلید `model_name` ثبت می‌گردد.

۳. فاز اول: CNN از صفر (آزمایش‌های ۰۱ تا ۰۹)

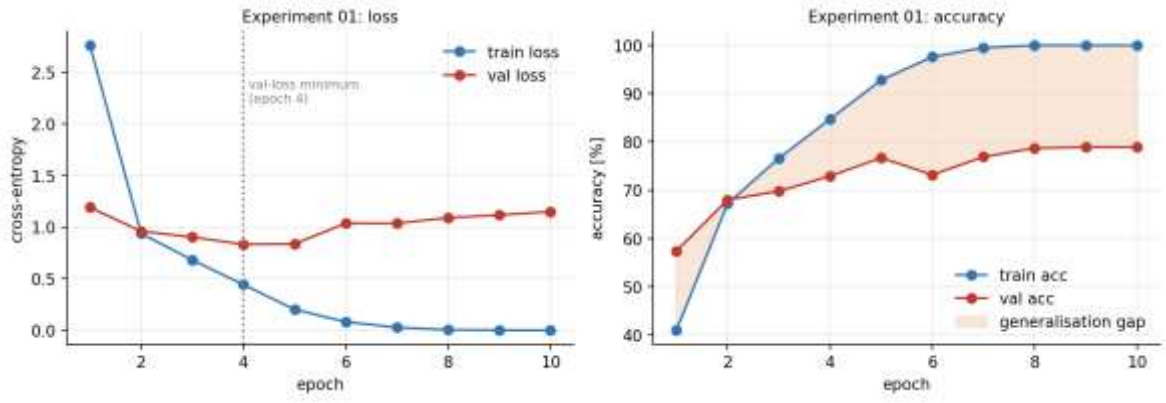
در این فاز یک شبکه ی کانولوشنی کوچک از ابتدا آموزش داده می‌شود و با «تغییر یک عامل در هر آزمایش» تلاش می‌شود بیش‌برازش کاهش یابد و دقت بالا برود. همه ی آزمایش‌ها ۱۰ اپوک (به‌جز GAP) اجرا شده‌اند و تنظیمات مشترک بخش ۲ را دارند.

۳.۱ آزمایش ۰۱: خط مبنا (Baseline CNN)

هدف: تعیین مرجعی که همه ی بهبودها با آن سنجیده می‌شود. این آزمایش هیچ افزایش داده، Dropout یا GAP ندارد و فقط نرمال‌سازی ImageNet را اعمال می‌کند.

جدول ۸ — لاگ آموزش آزمایش ۰۱ (ردیف زردرنگ: بهترین اپوک)

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۷۶۰۳	۴۱.۰٪	۱.۱۹۰۸	۵۷.۴٪
۲	۰.۹۳۷۹	۶۷.۲٪	۰.۹۵۹۹	۶۸.۰٪
۳	۰.۶۸۱۹	۷۶.۶٪	۰.۹۰۲۶	۶۹.۸٪
۴	۰.۴۴۲۱	۸۴.۷٪	۰.۸۳۲۹	۷۲.۹٪
۵	۰.۲۰۱۹	۹۲.۸٪	۰.۸۳۷۲	۷۶.۷٪
۶	۰.۰۸۲۴	۹۷.۶٪	۱.۰۳۶۷	۷۳.۲٪
۷	۰.۰۲۸۰	۹۹.۵٪	۱.۰۳۶۴	۷۶.۹٪
۸	۰.۰۰۵۸	۱۰۰.۰٪	۱.۰۹۰۵	۷۸.۷٪
۹	**۰.۰۰۱۷**	**۱۰۰.۰٪**	**۱.۱۱۶۴**	**۷۸.۹٪**
۱۰	۰.۰۰۰۹	۱۰۰.۰٪	۱.۱۴۹۶	۷۸.۹٪



شکل ۴ — منحنی‌های خطا و دقت خط مبنا: دقت آموزش به ۱۰۰٪ می‌رسد، اما خطای اعتبارسنجی پس از اپوک ۴ دوباره بالا می‌رود.

تحلیل. دقت آموزش در اپوک ۸ به ۱۰۰٪ می‌رسد، در حالی که بهترین دقت اعتبارسنجی ۷۸.۹۴٪ (اپوک ۹) است. شکاف آموزش/اعتبارسنجی برابر ۲۱.۱ = ٪۷۸.۹ − ٪۱۰۰ واحد درصد است. حداقل خطای اعتبارسنجی (≈ ۰.۸۳) در اپوک ۴–۵ رخ می‌دهد و پس از آن صعود می‌کند؛ این الگوی کلاسیک **بیش‌برازش** است: شبکه نمونه‌های آموزش را حفظ می‌کند و اطمینان بیش‌ازحد به پیش‌بینی‌های غلط پیدا می‌کند. جالب است که دقت اعتبارسنجی تا اپوک ۹ همچنان کمی بالا می‌رود، حتی وقتی خطا بدتر می‌شود؛ زیرا دقت فقط به علامت argmax وابسته است، نه به اطمینان (بخش ۲.۷).

جدول ۹ — گزارش طبقه‌بندی آزمایش ۰۱ (اعتبارسنجی، بهترین نقطه ی ذخیره)

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۸۴۲۹	۰.۸۰۸۲	۰.۸۲۵۲	۷۳
autobus	۰.۸۹۶۹	۰.۹۲۵۵	۰.۹۱۱۰	۹۴
kamyun	۰.۶۶۹۹	۰.۶۹۷۰	۰.۶۸۳۲	۹۹
kamyunet	۰.۶۳۱۶	۰.۶۵۹۳	۰.۶۴۵۲	۹۱
minibus	۰.۸۵۵۳	۰.۷۹۲۷	۰.۸۲۲۸	۸۲
savari	۰.۸۱۳۷	۰.۸۳۰۰	۰.۸۲۱۸	۱۰۰
taxi	۰.۹۱۸۴	۰.۹۰۹۱	۰.۹۱۳۷	۹۹
vanet	۰.۶۰۵۳	۰.۵۶۱۰	۰.۵۸۲۳	۴۱
accuracy			**۰.۷۸۹۴**	**۶۷۹**
macro avg	۰.۷۷۹۲	۰.۷۷۲۹	۰.۷۷۵۶	۶۷۹

جدول ۱۰ — ماتریس درهم‌ریختگی آزمایش ۰۱ (تعداد نمونه؛ سطرها: کلاس واقعی)

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۵۹	۱	۵	۲	۰	۱	۱	۴
autobus	۰	۸۷	۵	۱	۱	۰	۰	۰
kamyun	۲	۶	۶۹	۱۵	۳	۲	۱	۱
kamyunet	۶	۱	۱۳	۶۰	۶	۳	۰	۲
minibus	۰	۰	۶	۶	۶۵	۱	۴	۰
savari	۱	۲	۱	۵	۰	۸۳	۱	۷
taxi	۰	۰	۲	۲	۱	۳	۹۰	۱
vanet	۲	۰	۲	۴	۰	۹	۱	۲۳

تفسیر خطاها. بهترین کلاس‌ها taxi (F۱ = ۰.۹۱۴) و autobus (F۱ = ۰.۹۱۱) و ضعیف‌ترین vanet (F۱ = ۰.۵۸۲) و kamyunet (F۱ = ۰.۶۴۵) هستند. دو الگوی خطای پایدار دیده می‌شود که در تمام آزمایش‌های بعدی هم

تکرار می‌شود: **(الف)** ۱۵ نمونه ی (≈ ۱۵٪) kamyun به‌عنوان kamyunet و ۱۳ نمونه ی (≈ ۱۴٪) kamyunet به‌عنوان kamyun پیش‌بینی شده‌اند؛ **(ب)** ۹ از ۴۱ نمونه (≈ ۲۲٪). این خطاها «بزرگ‌ترین نشت‌های غیرقطری» هستند و تقریباً همیشه از شباهت اندازه و شکل ناشی می‌شوند.

> **نتیجه کلیدی: نتیجه ی آزمایش ۰۱:** خط مبنا ۷۸.۹۴٪ دقت دارد و شکاف ۲۱.۱ واحد درصدی نشان می‌دهد مشکل اصلی **تعمیم** است، نه یادگیری داده‌های آموزش.

۳.۲ آزمایش ۰۲: افزایش داده (Data Augmentation)

فرضیه: با تولید تغییرات تصادفی «واقع‌گرایانه» از هر تصویر، شبکه نمی‌تواند تصاویر را حفظ کند و بیش‌برازش کم می‌شود. فقط تبدیل **آموزش** تغییر می‌کند: `RandomHorizontalFlip(p=0.5)`، `RandomRotation(10)` و `ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)`.

مبنای ریاضی. افزایش داده معادل جایگزین کردن ریسک تجربی با امید ریاضی روی یک خانواده ی تبدیل‌هاست: به‌جای کمینه‌سازی $\frac{1}{N}\sum_i \mathcal{L}\left(f(x_i), y_i\right)$ کمینه‌سازی زیر انجام می‌شود که در آن $T \sim \mathcal{T}$ یک تبدیل تصادفی است و فرض می‌کنیم برچسب تحت T ثابت می‌ماند:

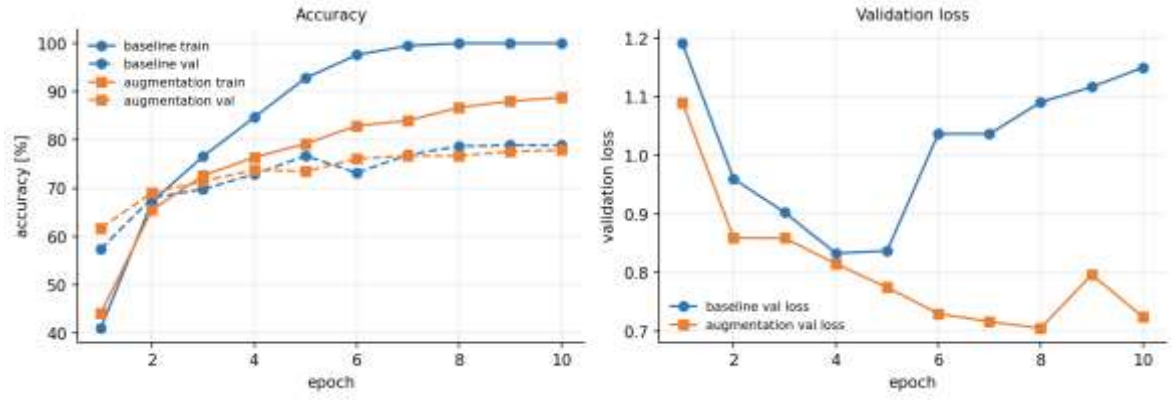
$$\min_{\theta} \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}_{T\sim\mathcal{T}}\left[\mathcal{L}(f_\theta(T(x_i)), y_i)\right]$$

چرخش $\pm 10^\circ$ با ماتریس چرخش R_ϕ و $\phi \sim U(-10^\circ, 10^\circ)$ اعمال می‌شود؛ ColorJitter هر کانال را در ضریب تصادفی از بازه ی $[0.8, 1.2]$ تغییر می‌دهد. هر نمونه در هر اپوک «نسخه ی تازه‌ای» می‌بیند، پس ظرفیت حفظ کردن عملاً پایین می‌آید.

جدول ۱۱ — لاگ آموزش آزمایش ۰۲ (افزایش داده)

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۰۶۴۲	۴۴.۱٪	۱.۰۸۹۷	۶۱.۷٪
۲	۱.۰۰۴۳	۶۵.۵٪	۰.۸۵۹۱	۶۹.۱٪
۳	۰.۸۱۶۲	۷۲.۶٪	۰.۸۵۸۸	۷۱.۴٪
۴	۰.۶۸۹۶	۷۶.۴٪	۰.۸۱۵۱	۷۳.۸٪
۵	۰.۶۰۳۱	۷۹.۲٪	۰.۷۷۴۷	۷۳.۵٪

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۶	۰.۵۰۲۴	۸۲.۹٪	۰.۷۳۰۰	۷۶.۱٪
۷	۰.۴۵۴۸	۸۴.۰٪	۰.۷۱۶۵	۷۶.۷٪
۸	۰.۳۹۹۸	۸۶.۷٪	۰.۷۰۵۲	۷۶.۷٪
۹	۰.۳۳۳۳	۸۸.۰٪	۰.۷۹۶۶	۷۷.۶٪
۱۰	**۰.۳۰۵۹**	**۸۸.۸٪**	**۰.۷۲۴۴**	**۷۷.۹٪**



شکل ۵ — مقایسه ی خط مبنا (آبی) و افزایش داده (نارنجی): شکاف دقت کوچک‌تر و خطای اعتبارسنجی پایین‌تر و پایدارتر است.

تحلیل. هر دو منحنی تا اپوک ۱۰ بهبود می‌یابند و واگرایی تند خط مبنا دیده نمی‌شود. شکاف از ۲۱.۱ به — ۸۸.۸ ۱۰.۸ = ۷۷.۹ واحد درصد نصف شده و خطای اعتبارسنجی از ۱.۱۱۶ به ۰.۷۲۴ (حدود ۳۵٪ کاهش) رسیده است. اما بهترین دقت اعتبارسنجی ۷۷.۹۱٪ است، یعنی حدود یک واحد درصد **کمتر** از خط مبنا (۷۸.۹۴٪). یعنی افزایش داده تعمیم را «پایدارتر» کرده ولی دقت را بهبود نداده است.

جدول ۱۲ — گزارش طبقه‌بندی آزمایش ۰۲

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۸۲۵۰	۰.۹۰۴۱	۰.۸۶۲۷	۷۳
autobus	۰.۸۷۶۳	۰.۹۰۴۳	۰.۸۹۰۱	۹۴
kamyun	۰.۵۸۷۷	۰.۶۷۶۸	۰.۶۲۹۱	۹۹
kamyunet	۰.۶۷۱۲	۰.۵۳۸۵	۰.۵۹۷۶	۹۱
minibus	۰.۸۱۲۵	۰.۷۹۲۷	۰.۸۰۲۵	۸۲
savari	۰.۷۵۸۹	۰.۸۵۰۰	۰.۸۰۱۹	۱۰۰
taxi	۱.۰۰۰۰	۰.۹۲۹۳	۰.۹۶۳۴	۹۹
vanet	۰.۶۴۵۲	۰.۴۸۷۸	۰.۵۵۵۶	۴۱
accuracy			**۰.۷۷۹۱**	**۶۷۹**
macro avg	۰.۷۷۲۱	۰.۷۶۰۴	۰.۷۶۲۸	۶۷۹

جدول ۱۳ — ماتریس درهم‌ریختگی آزمایش ۰۲

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۶۶	۱	۲	۱	۰	۱	۰	۲
autobus	۱	۸۵	۵	۱	۱	۰	۰	۱
kamyun	۲	۸	۶۷	۱۱	۷	۱	۰	۳
kamyunet	۵	۱	۲۴	۴۹	۶	۶	۰	۰
minibus	۰	۲	۱۰	۵	۶۵	۰	۰	۰
savari	۳	۰	۲	۴	۱	۸۵	۰	۵
taxi	۰	۰	۳	۰	۰	۴	۹۲	۰
vanet	۳	۰	۱	۲	۰	۱۵	۰	۲۰

تفسیر. taxi به دقت (Precision) کامل ۱.۰ رسیده (با کمی کاهش Recall)، ولی `kamyunet → kamyun` از ۱۳ نمونه در خط مبنا به **۲۴ نمونه** افزایش یافته است. یعنی افزایش داده شکاف آموزش/اعتبارسنجی را کم کرده اما اشتباهات مبتنی بر اندازه را «جابه‌جا» کرده است نه حذف. توجیه محتمل: تبدیل‌های رنگی و چرخش اطلاعات اندازه ی نسبی را که برای تفکیک kamyun/kamyunet لازم است تغییر نمی‌دهند و این کلاس‌ها به ویژگی‌های دقیق‌تری نیاز دارند که یک CNN کوچک ندارد.

جدول ۱۴ — خلاصه ی مقایسه ی خط مبنا و افزایش داده (از سلول مقایسه ی نوت‌بوک)

	بهترین اپوک	دقت اعتبارسنجی	خطای اعتبارسنجی
خط مبنا	۹	۰.۷۸۹۴	۱.۱۱۶۴
افزایش داده	۱۰	۰.۷۷۹۱	۰.۷۲۴۴

نتیجه کلیدی: نتیجه ی آزمایش ۰۲: افزایش داده شکاف را نصف و خطای اعتبارسنجی را به‌شدت کم کرد، ولی دقت را بهبود نداد (۷۷.۹۱٪ در برابر ۷۸.۹۴٪). برای CNN کوچک، افزایش داده «پایداری» می‌دهد نه «دقت بیشتر».

۳.۳ آزمایش ۰۳: Dropout

فرضیه: Dropout هم‌تطبیقی (Co-adaptation) نورون‌ها در لایه ی پرپارامتر FC را می‌شکند و بیش‌برازش را کم می‌کند. سه احتمال $p \in \{۰.۲, ۰.۳, ۰.۵\}$ با تبدیل ساده (بدون افزایش داده) مقایسه می‌شود.

مبنای ریاضی (Inverted Dropout). در آموزش، برای هر نورون یک ماسک $m_i \sim \text{Bernoulli}(۱-p)$ نمونه‌برداری و خروجی با $۱/(۱-p)$ مقیاس می‌شود تا امید ریاضی حفظ شود؛ در استنتاج هیچ ماسکی اعمال نمی‌شود:

$$\tilde{h}_i = \frac{m_i\, h_i}{۱-p}, \qquad \mathbb{E}\big[\tilde{h}_i\big] = h_i$$

Dropout را می‌توان «میانگین‌گیری تقریبی از تعداد نمایی زیرشبکه» تفسیر کرد. هرچه p بزرگ‌تر باشد، نوفه ی آموزش بیشتر و سرعت یادگیری کمتر است (چنان‌که در لاگ $p = ۰.۵$ دقت اپوک ۱ فقط ۴۵.۴٪ است).

لاگ‌های آموزش

جدول ۱۵ — لاگ آموزش با dropout = ۰.۲

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۸۷۰۷	۳۸.۹٪	۱.۲۵۳۳	۵۷.۴٪
۲	۱.۰۵۸۵	۶۳.۵٪	۰.۹۸۲۳	۶۵.۰٪
۳	۰.۸۰۷۷	۷۲.۱٪	۰.۹۴۸۶	۶۵.۸٪
۴	۰.۵۸۸۱	۷۹.۸٪	۰.۸۰۴۰	۷۳.۳٪
۵	۰.۴۴۸۰	۸۵.۱٪	۰.۸۷۶۱	۷۳.۶٪
۶	۰.۳۱۲۹	۸۸.۸٪	۰.۸۴۳۶	۷۴.۲٪
۷	۰.۲۷۱۴	۹۱.۴٪	۰.۹۵۴۸	۷۳.۱٪
۸	۰.۱۶۳۱	۹۴.۵٪	۰.۹۹۰۸	۷۴.۵٪
۹	**۰.۱۱۹۶**	**۹۶.۴٪**	**۱.۰۷۵۰**	**۷۵.۶٪**
۱۰	۰.۰۸۴۱	۹۷.۲٪	۱.۲۱۰۰	۷۴.۱٪

جدول ۱۶ — لاگ آموزش با dropout = ۰.۳

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۵۱۹۱	۳۸.۲٪	۱.۱۶۹۳	۵۷.۱٪
۲	۱.۰۳۲۷	۶۴.۶٪	۰.۸۹۲۴	۶۹.۴٪
۳	۰.۷۰۵۹	۷۵.۹٪	۰.۷۹۶۱	۷۳.۱٪
۴	۰.۵۰۷۸	۸۲.۵٪	۰.۷۹۶۴	۷۴.۱٪
۵	۰.۳۸۲۷	۸۶.۵٪	۰.۷۵۶۰	۷۵.۶٪
۶	۰.۲۶۵۸	۹۱.۰٪	۰.۸۸۸۴	۷۵.۰٪
۷	۰.۱۷۸۳	۹۳.۶٪	۰.۸۸۵۴	۷۵.۴٪
۸	۰.۱۴۸۱	۹۵.۵٪	۱.۰۱۳۳	۷۶.۱٪

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۹	**۰.۱۰۳۲**	**۹۶.۶٪**	**۰.۹۳۲۶**	**۷۷.۰٪**
۱۰	۰.۰۹۲۴	۹۷.۲٪	۱.۰۸۰۱	۷۳.۵٪

جدول ۱۷ — لاگ آموزش با dropout = ۰.۵

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۵۷۶۱	۲۹.۱٪	۱.۵۶۲۲	۴۵.۴٪
۲	۱.۳۶۹۵	۵۲.۲٪	۱.۱۳۵۸	۶۱.۳٪
۳	۰.۹۸۴۹	۶۵.۹٪	۰.۹۲۲۳	۶۸.۵٪
۴	۰.۷۶۰۳	۷۳.۷٪	۰.۸۸۵۹	۶۹.۲٪
۵	۰.۶۰۱۵	۷۹.۱٪	۰.۷۶۳۶	۷۲.۳٪
۶	۰.۵۱۵۹	۸۲.۲٪	۰.۷۸۰۶	۷۲.۸٪
۷	۰.۳۸۱۶	۸۶.۷٪	۰.۸۱۰۵	۷۴.۷٪
۸	۰.۳۲۳۴	۸۸.۵٪	۰.۸۲۳۰	۷۳.۵٪
۹	۰.۲۹۳۳	۸۹.۵٪	۰.۸۵۲۶	۷۴.۵٪
۱۰	**۰.۲۳۹۴**	**۹۱.۲٪**	**۰.۸۸۰۹**	**۷۵.۰٪**

جدول ۱۸ — مقایسه ی سه مقدار Dropout

Dropout	بهترین اپوک	بهترین دقت اعتبارسنجی	دقت آموزش در بهترین اپوک	شکاف آموزش/اعتبارسنجی
۰.۲	۹	۰.۷۵۵۵	۰.۹۶۳۵	۰.۲۰۸۰
۰.۳	**۹**	**۰.۷۷۰۳**	**۰.۹۶۶۵**	**۰.۱۹۶۲**
۰.۵	۱۰	۰.۷۴۹۶	۰.۹۱۱۵	۰.۱۶۱۹

تحلیل. $p = ۰.۳$ بهترین دقت اعتبارسنجی (۷۷.۰۳٪) را می‌دهد؛ $p = ۰.۵$ کوچک‌ترین شکاف (۰.۱۶۱۹) را دارد اما کمترین دقت (۷۴.۹۶٪) را؛ چون Dropout زیاد دقت آموزش را هم پایین می‌آورد (۹۱.۱۵٪ در برابر ۹۶.۶۵٪). $p = ۰.۲$ نیز شکاف ۰.۲۰۸۰ و دقت ۷۵.۵۵٪ دارد. رابطه بین p و دقت **یکنوا نیست**: مقدار میانه بهترین بود. با این حال همه ی مقادیر Dropout از خط مبنا (۷۸.۹۴٪) بدتر ماندند؛ بنابراین Dropout در این معماری و تنظیم، سودمند نبود. تفاوت‌های ۱–۲ واحد درصدی بین سه مقدار نیز در حد نوسان تصادفی یک اجرای واحد است (بخش ۳.۵).

جدول ۱۹ — گزارش طبقه‌بندی آزمایش ۰۳ (dropout = ۰.۳)

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۷۹۴۵	۰.۷۹۴۵	۰.۷۹۴۵	۷۳
autobus	۰.۹۱۳۰	۰.۸۹۳۶	۰.۹۰۳۲	۹۴
kamyun	۰.۶۴۷۷	۰.۵۷۵۸	۰.۶۰۹۶	۹۹
kamyunet	۰.۶۰۶۱	۰.۶۵۹۳	۰.۶۳۱۶	۹۱
minibus	۰.۷۸۵۷	۰.۸۰۴۹	۰.۷۹۵۲	۸۲
savari	۰.۷۶۹۹	۰.۸۷۰۰	۰.۸۱۶۹	۱۰۰
taxi	۰.۹۳۸۸	۰.۹۲۹۳	۰.۹۳۴۰	۹۹
vanet	۰.۵۹۳۸	۰.۴۶۳۴	۰.۵۲۰۵	۴۱
accuracy			**۰.۷۷۰۳**	**۶۷۹**
macro avg	۰.۷۵۶۲	۰.۷۴۸۹	۰.۷۵۰۷	۶۷۹

جدول ۲۰ — ماتریس درهم‌ریختگی آزمایش ۰۳ (dropout = ۰.۳)

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۵۸	۰	۲	۳	۵	۴	۱	۰
autobus	۱	۸۴	۷	۱	۰	۰	۱	۰
kamyun	۲	۸	۵۷	۲۳	۵	۱	۲	۱
kamyunet	۷	۰	۱۲	۶۰	۸	۲	۱	۱
minibus	۲	۰	۶	۶	۶۶	۲	۰	۰
savari	۱	۰	۱	۲	۰	۸۷	۰	۹
taxi	۰	۰	۱	۰	۰	۴	۹۲	۲
vanet	۲	۰	۲	۴	۰	۱۳	۱	۱۹

در این ماتریس `kamyun → kamyunet` به **۲۳ نمونه** (≈ ۲۳٪) می‌رسد که بزرگ‌ترین مقدار تا این نقطه است؛ و `vanet → savari` برابر ۱۳ نمونه از ۴۱ (≈ ۳۲٪) است. پس Dropout در این سطح دو الگوی خطای پایدار را حل نمی‌کند.

نتیجه کلیدی: **نتیجه ی آزمایش ۰۳:** بهترین مقدار $p = ۰.۳$ با ۷۷.۰۳٪ است؛ اما هیچ مقدار Dropout از خط مبنا بهتر نشد و شکاف آموزش/اعتبارسنجی هنوز بزرگ است.

۳.۴ آزمایش ۰۴: Max Pooling در برابر Average Pooling

هدف: مقایسه ی دو نوع ادغام (Pooling). در `SimpleCNNAvgPool` هر `MaxPool2d` با `AvgPool2d` جایگزین می‌شود و بقیه ی تنظیمات (بدون Dropout و افزایش داده) مثل خط مبنا است.

مبنای ریاضی. برای پنجره ی Ω (اینجا ۲ × ۲):

$$y_{\max} = \max_{(i,j)\in\Omega} x_{ij}, \qquad y_{\text{avg}} = \frac{1}{|\Omega|}\sum_{(i,j)\in\Omega} x_{ij}$$

Max Pooling فقط قوی‌ترین پاسخ (حضور یک ویژگی) را عبور می‌دهد و گرادیان را فقط به همان پیکسل می‌فرستد؛ نسبت به جابه‌جایی‌های کوچک پایاست و ویژگی‌های تیز (لبه‌ها) را حفظ می‌کند. Average Pooling همه را هموار می‌کند و گرادیان را به‌طور مساوی (۱/۴) بین چهار پیکسل پخش می‌کند؛ بافت‌ها را خوب منتقل می‌کند ولی لبه‌ها را محو می‌کند.

جدول ۲۱ — لاگ آموزش آزمایش ۰۴ (Average Pooling)

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۶۳۳۱	۳۸.۳٪	۱.۲۲۵۷	۶۰.۱٪
۲	۰.۹۹۱۴	۶۵.۹٪	۰.۹۷۲۷	۶۶.۱٪
۳	۰.۶۷۶۹	۷۶.۶٪	۰.۸۹۰۸	۷۱.۴٪
۴	۰.۴۶۰۹	۸۴.۶٪	۰.۹۱۶۵	۷۲.۳٪
۵	**۰.۳۳۵۰**	**۸۸.۵٪**	**۰.۹۵۳۳**	**۷۲.۵٪**
۶	۰.۲۲۸۳	۹۲.۳٪	۱.۰۵۲۱	۷۱.۰٪
۷	۰.۱۲۴۱	۹۶.۲٪	۱.۲۱۸۲	۷۱.۶٪
۸	۰.۰۶۳۴	۹۸.۱٪	۱.۳۷۹۵	۶۹.۵٪
۹	۰.۰۷۳۵	۹۷.۹٪	۱.۶۲۰۹	۷۱.۴٪
۱۰	۰.۰۴۵۸	۹۸.۳۸٪	۱.۵۲۲۴	۷۲.۴۶٪

تحلیل لاگ. بهترین دقت اعتبارسنجی حدود ۷۲.۵٪ (اپوک ۵؛ در `Result.json` برابر ۷۲.۴۵٪) است. پس از آن دقت آموزش تا ۹۸٪ بالا می‌رود ولی خطای اعتبارسنجی تا ۱.۶۲ افزایش می‌یابد؛ بیش‌برازش در Average Pooling **شدیدتر** از خط مبنا دیده می‌شود.

جدول ۲۲ — گزارش طبقه‌بندی ثبت‌شده در نوت‌بوک آزمایش ۰۴

کلاس	Precision	Recall	F1	Support
ambulance	۰.۶۹۳۲	۰.۸۴۷۲	۰.۷۶۲۵	۷۲
autobus	۰.۸۸۱۲	۰.۹۴۶۸	۰.۹۱۲۸	۹۴

کلاس	Precision	Recall	F۱	Support
kamyun	۰.۶۳۱۱	۰.۶۵۶۶	۰.۶۴۳۶	۹۹
kamyunet	۰.۶۳۱۰	۰.۵۸۲۴	۰.۶۰۵۷	۹۱
minibus	۰.۹۰۷۷	۰.۷۱۹۵	۰.۸۰۲۷	۸۲
savari	۰.۷۴۱۴	۰.۸۶۰۰	۰.۷۹۶۳	۱۰۰
taxi	۰.۹۲۰۸	۰.۹۳۹۴	۰.۹۳۰۰	۹۹
vanet	۰.۸۹۹۲	۰.۷۷۳۳	۰.۸۳۱۵	۱۵۰
accuracy			**۰.۷۹۰۳**	**۷۸۷**
macro avg	۰.۷۸۸۲	۰.۷۹۰۷	۰.۷۸۵۶	۷۸۷

جدول ۲۳ — ماتریس درهم‌ریختگی ثبت‌شده در نوت‌بوک آزمایش ۰۴

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۶۰	۱	۴	۲	۰	۱	۱	۴
autobus	۰	۸۹	۴	۰	۱	۰	۱	۰
kamyun	۲	۶	۶۶	۱۸	۳	۲	۱	۱
kamyunet	۵	۱	۱۴	۶۱	۶	۲	۰	۲
minibus	۰	۰	۷	۶	۶۴	۱	۴	۰
savari	۱	۳	۰	۵	۰	۸۲	۱	۸
taxi	۰	۰	۲	۲	۱	۳	۹۰	۱
vanet	۲	۰	۲	۳	۰	۱۱	۱	۲۲

جدول ۲۴ — سلول مقایسه ی نهایی Max در برابر Avg (آن‌گونه که چاپ شده است)

	بهترین اپوک	بهترین دقت اعتبارسنجی	خطای اعتبارسنجی در بهترین اپوک
Max Pooling (baseline, from experiment_results.json)	۶	۰.۷۸۹۳	۱.۱۱۶
Avg Pooling	۵	۰.۷۲۵۰	۰.۹۵۳۳

نتیجه کلیدی: **نتیجه ی آزمایش ۰۴:** با اتکا به لاگ آموزش و فایل نتایج، (۷۲.۴۵٪) Average Pooling از (۷۸.۹۴٪) Max Pooling حدود ۶.۵ واحد درصد ضعیف‌تر است. Max Pooling انتخاب پیش‌فرض و مناسب‌تر برای این مسئله است. (نکته ی گزارش منبع هم این است که ترتیب‌بندی بر اساس دقت و بر اساس خطا ممکن است متفاوت باشد.)

۳.۵ آزمایش ۰۵: Weight Decay

هدف: سنجش اثر منظم‌سازی وزن. برای جداسازی اثر، بهینه‌ساز از Adam به **AdamW** تغییر یافت و $\lambda = ۰$ با $\lambda = ۱۰^{-۴}$ مقایسه شد؛ بقیه ی عوامل مانند خط مبنا است.

تحلیل انتقادی: با این محاسبه، $\lambda = ۱۰^{-۴}$ در AdamW با نرخ یادگیری $۱۰^{-۳}$ و ۱۰ اپوک عملاً منظم‌ساز **بسیار ضعیفی** است. بنابراین بخش عمده ی تفاوت مشاهده‌شده بین دو اجرا (حدود ۲ واحد درصد) احتمالاً از «واگرایی مسیر آموزش» و نوسان تصادفی ناشی می‌شود، نه از اثر واقعی منظم‌سازی. شاهد دیگر: حتی AdamW با $\lambda = ۰$ دقتی (۷۶.۱۴٪) پایین‌تر از Adam خط مبنا (۷۸.۹۴٪) دارد؛ یعنی صرفِ تغییر بهینه‌ساز/مسیر، حدود ۲.۸ واحد درصد جابه‌جایی ایجاد کرده است.

جدول ۲۵ — لاگ آموزش با (AdamW) weight_decay = ۰

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۵۶۸۵	۲۸.۲٪	۱.۵۴۲۹	۴۱.۴٪
۲	۱.۱۵۸۱	۵۸.۹٪	۱.۲۴۶۱	۶۱.۶٪
۳	۰.۵۶۱۵	۸۱.۳٪	۰.۸۱۱۶	۷۳.۲٪
۴	۰.۲۴۵۰	۹۲.۴٪	۰.۸۸۸۷	۷۲.۹٪
۵	۰.۰۹۴۴	۹۷.۶٪	۱.۰۴۶۲	۷۴.۷٪
۶	۰.۰۴۲۶	۹۸.۸٪	۱.۱۵۷۳	۷۵.۶٪
۷	۰.۰۱۴۶	۹۹.۷٪	۱.۲۳۶۱	۷۵.۴٪
۸	۰.۰۰۲۳	۱۰۰.۰٪	۱.۲۵۹۴	۷۵.۶٪
۹	۰.۰۰۱۳	۱۰۰.۰٪	۱.۳۱۲۴	۷۵.۹٪
۱۰	**۰.۰۰۰۸**	**۱۰۰.۰٪**	**۱.۳۴۵۲**	**۷۶.۱٪**

جدول ۲۶ — لاگ آموزش با (AdamW) weight_decay = ۱e-۴

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۲.۶۱۲۱	۳۶.۵٪	۱.۲۴۸۴	۵۴.۹٪
۲	۰.۹۵۰۵	۶۶.۸٪	۰.۹۵۱۹	۶۸.۲٪
۳	۰.۶۰۸۳	۷۸.۴٪	۰.۸۶۶۰	۷۰.۰٪
۴	۰.۳۶۷۷	۸۷.۰٪	۰.۸۵۴۹	۷۵.۶٪
۵	۰.۲۳۵۸	۹۲.۹٪	۰.۸۹۷۵	۷۷.۵٪
۶	۰.۱۱۹۲	۹۶.۰٪	۰.۹۶۷۸	۷۵.۴٪

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۷	۰.۰۴۶۱	۹۹.۰٪	۱.۰۵۲۸	۷۶.۷٪
۸	۰.۰۱۸۹	۹۹.۶٪	۱.۰۷۵۸	۷۷.۹٪
۹	۰.۰۰۶۲	۱۰۰.۰٪	۱.۰۸۴۹	۷۸.۱٪
۱۰	**۰.۰۰۲۱**	**۱۰۰.۰٪**	**۱.۱۴۱۸**	**۷۸.۲٪**

هر دو پیکربندی تا اپوک ۸–۹ به دقت آموزش ۱۰۰٪ می‌رسند؛ پس $\lambda = 10^{-4}$ مانع حفظ کامل نمی‌شود. با این حال دقت اعتبارسنجی اجرای 10^{-4} در ۸ از ۱۰ اپوک (همه به‌جز اپوک‌های ۳ و ۶) بالاتر از اجرای بدون decay است و خطای اعتبارسنجی آن در ۹ از ۱۰ اپوک (همه به‌جز اپوک ۳) کمتر است.

جدول ۲۷ — گزارش طبقه‌بندی weight_decay = ۰ (بهترین اپوک ۱۰، دقت ۰.۷۶۱۴)

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۸۳۰۸	۰.۷۳۹۷	۰.۷۸۲۶	۷۳
autobus	۰.۸۷۱۰	۰.۸۶۱۷	۰.۸۶۶۳	۹۴
kamyun	۰.۵۸۵۶	۰.۶۵۶۶	۰.۶۱۹۰	۹۹
kamyunet	۰.۶۴۴۴	۰.۶۳۷۴	۰.۶۴۰۹	۹۱
minibus	۰.۷۸۵۷	۰.۸۰۴۹	۰.۷۹۵۲	۸۲
savari	۰.۷۶۷۲	۰.۸۹۰۰	۰.۸۲۴۱	۱۰۰
taxi	۰.۹۴۶۸	۰.۸۹۹۰	۰.۹۲۲۳	۹۹
vanet	۰.۵۷۶۹	۰.۳۶۵۹	۰.۴۴۷۸	۴۱
accuracy			**۰.۷۶۱۴**	**۶۷۹**
macro avg	۰.۷۵۱۱	۰.۷۳۱۹	۰.۷۳۷۳	۶۷۹

جدول ۲۸ — گزارش طبقه‌بندی weight_decay = ۱e-۴ (بهترین اپوک ۱۰، دقت ۰.۷۸۲۰)

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۸۰۲۸	۰.۷۸۰۸	۰.۷۹۱۷	۷۳
autobus	۰.۸۷۵۰	۰.۸۹۳۶	۰.۸۸۴۲	۹۴
kamyun	۰.۶۲۵۰	۰.۶۵۶۶	۰.۶۴۰۴	۹۹
kamyunet	۰.۶۷۰۶	۰.۶۲۶۴	۰.۶۴۷۷	۹۱
minibus	۰.۸۰۴۹	۰.۸۰۴۹	۰.۸۰۴۹	۸۲
savari	۰.۸۲۸۶	۰.۸۷۰۰	۰.۸۴۸۸	۱۰۰
taxi	۰.۹۴۹۵	۰.۹۴۹۵	۰.۹۴۹۵	۹۹
vanet	۰.۵۶۷۶	۰.۵۱۲۲	۰.۵۳۸۵	۴۱

کلاس	Precision	Recall	F۱	Support
accuracy			**۰.۷۸۲۰**	**۶۷۹**
macro avg	۰.۷۶۵۵	۰.۷۶۱۷	۰.۷۶۳۲	۶۷۹

جدول ۲۹ — ماتریس درهم‌ریختگی — weight_decay = ۰

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۵۴	۰	۶	۲	۳	۱	۰	۷
autobus	۰	۸۱	۶	۲	۱	۲	۲	۰
kamyun	۲	۷	۶۵	۱۶	۷	۱	۰	۱
kamyunet	۴	۰	۲۲	۵۸	۵	۲	۰	۰
minibus	۱	۲	۷	۴	۶۶	۲	۰	۰
savari	۰	۱	۲	۲	۱	۸۹	۲	۳
taxi	۰	۲	۱	۱	۱	۵	۸۹	۰
vanet	۴	۰	۲	۵	۰	۱۴	۱	۱۵

جدول ۳۰ — ماتریس درهم‌ریختگی — weight_decay = ۱e-۴

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۵۷	۱	۸	۱	۲	۱	۰	۳
autobus	۲	۸۴	۴	۰	۲	۰	۱	۱
kamyun	۱	۸	۶۵	۱۷	۴	۱	۱	۲
kamyunet	۵	۱	۱۷	۵۷	۵	۴	۰	۲
minibus	۱	۱	۵	۵	۶۶	۳	۱	۰
savari	۰	۰	۲	۲	۱	۸۷	۱	۷
taxi	۰	۰	۱	۰	۲	۱	۹۴	۱
vanet	۵	۱	۲	۳	۰	۸	۱	۲۱

مقایسه ی Recall: در اجرای $\lambda = ۱۰^{-۴}$ چهار کلاس بهتر شده‌اند (ambulance از ۰.۷۴۰ به ۰.۷۸۱، autobus از ۰.۸۶۲ به ۰.۸۹۴، taxi از ۰.۸۹۹ به ۰.۹۵۰ و vanet از ۰.۳۶۶ به ۰.۵۱۲)، دو کلاس تغییر نکرده‌اند (kamyun و minibus) و دو کلاس اندکی افت کرده‌اند (kamyunet از ۰.۶۳۷ به ۰.۶۲۶ و savari از ۰.۸۹۰ به ۰.۸۷۰). برجسته‌ترین بهبود vanet است؛ نشت `vanet → savari` از ۱۴ به ۸ نمونه و `kamyunet → kamyun` از ۲۲ به ۱۷ نمونه رسیده، ولی `kamyun → kamyunet` تقریباً ثابت (۱۶ در برابر ۱۷) مانده است. به هر حال با توجه به تحلیل بالا، این بهبود را باید با احتیاط تفسیر کرد.

جدول ۳۱ — سلول مقایسه ی نهایی آزمایش ۰۵

	بهترین اپوک	بهترین دقت اعتبارسنجی	خطای اعتبارسنجی در بهترین اپوک
Baseline	۹	۰.۷۸۹۴	۱.۱۱۶۴
weight_decay = ۰	۱۰	۰.۷۶۱۴	۱.۳۴۵۲
weight_decay = ۱e-۴	۱۰	۰.۷۸۲۰	۱.۱۴۱۸

نتیجه کلیدی: نتیجه ی آزمایش ۰۵: weight decay برابر $۱۰^{-۴}$ دقت را از ۷۶.۱۴٪ به ۷۸.۲۰٪ رسانده و به خط مبنا (۷۸.۹۴٪) نزدیک شده است، اما از نظر ریاضی انقباض وزن بسیار ناچیز است؛ پس باید این بهبود را در محدوده ی نوسان تصادفی دانست.

۳.۶ آزمایش ۰۶: زمان‌بند نرخ یادگیری (ReduceLROnPlateau)

هدف: آیا کاهش تطبیقی نرخ یادگیری در حوالی «کفه ی» بهبود به همگرایی بهتر می‌انجامد؟ نرخ یادگیری با ۰.۰۰۱ شروع می‌شود و وقتی خطای اعتبارسنجی برای چند اپوک بهبود نیابد، در یک ضریب ضرب می‌شود:

$$\eta_{t+۱} = \begin{cases} \gamma\,\eta_t & \text{اگر } \mathcal{L}_{val} \text{ بیش از } P \text{ اپوک بهبود نیافته باشد} \\ \eta_t & \text{otherwise} \end{cases}$$

در `Result.json` پارامترهای زمان‌بند ضریب $\gamma = ۰.۵$ و صبر $P = ۲$ ثبت شده‌اند. دلیل شهودی: نرخ یادگیری بزرگ در ابتدا جست‌وجوی سریع می‌کند، و نرخ کوچک در انتها به شبکه اجازه می‌دهد با گام‌های ظریف‌تر در دره‌ای باریک جا بگیرد.

جدول ۳۲ — لاگ آموزش آزمایش ۰۶ (اپوک ۸ بهترین است)

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی	نرخ یادگیری
۱	۲.۷۶۰۳	۴۱.۰٪	۱.۱۹۰۸	۵۷.۴٪	۰.۰۰۱۰۰۰
۲	۰.۹۳۷۹	۶۷.۲٪	۰.۹۵۹۹	۶۸.۰٪	۰.۰۰۱۰۰۰
۳	۰.۶۸۱۹	۷۶.۶٪	۰.۹۰۲۶	۶۹.۸٪	۰.۰۰۱۰۰۰
۴	۰.۴۴۲۱	۸۴.۷٪	۰.۸۳۲۹	۷۲.۹٪	۰.۰۰۱۰۰۰
۵	۰.۲۰۱۹	۹۲.۸٪	۰.۸۳۷۲	۷۶.۷٪	۰.۰۰۱۰۰۰
۶	۰.۰۸۲۴	۹۷.۶٪	۱.۰۳۶۷	۷۳.۲٪	۰.۰۰۱۰۰۰
۷	۰.۰۲۲۰	۹۹.۶٪	۰.۹۷۶۷	۷۸.۵٪	۰.۰۰۰۵۰۰
۸	**۰.۰۰۵۶**	**۱۰۰.۰٪**	**۱.۰۲۰۷**	**۷۸.۶٪**	**۰.۰۰۰۵۰۰**

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی	نرخ یادگیری
۹	۰.۰۰۳۲	۱۰۰.۰٪	۱.۰۳۹۳	۷۸.۵٪	۰.۰۰۰۲۵۰
۱۰	۰.۰۰۲۶	۱۰۰.۰٪	۱.۰۵۶۳	۷۸.۶٪	۰.۰۰۰۲۵۰

نرخ یادگیری در اپوک ۷ به ۰.۰۰۰۵ و در اپوک ۹ به ۰.۰۰۰۲۵ کاهش یافته است. با وجود این کاهش‌ها دقت آموزش به ۱۰۰٪ می‌رسد و دقت اعتبارسنجی حوالی ۷۸–۷۹٪ می‌ماند: شکاف بزرگ پایدار است. خطای اعتبارسنجی کمینه ی خود (۰.۸۳۲۹) را در اپوک ۴ دارد و سپس بالا می‌رود، در حالی که دقت اعتبارسنجی تا اپوک ۸ بهتر می‌شود؛ دوباره نشان می‌دهد دقت و خطا همزمان به اوج نمی‌رسند.

جدول ۳۳ — گزارش طبقه‌بندی آزمایش ۰۶ (بهترین دقت ۷۸.۶۵٪ در اپوک ۸، ۶۷۹ نمونه)

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۸۵۷۱	۰.۸۲۱۹	۰.۸۳۹۲	۷۳
autobus	۰.۸۹۰۰	۰.۹۴۶۸	۰.۹۱۷۵	۹۴
kamyun	۰.۶۶۶۷	۰.۶۶۶۷	۰.۶۶۶۷	۹۹
kamyunet	۰.۶۲۸۹	۰.۶۷۰۳	۰.۶۴۸۹	۹۱
minibus	۰.۸۵۳۳	۰.۷۸۰۵	۰.۸۱۵۳	۸۲
savari	۰.۸۰۳۹	۰.۸۲۰۰	۰.۸۱۱۹	۱۰۰
taxi	۰.۹۱۸۴	۰.۹۰۹۱	۰.۹۱۳۷	۹۹
vanet	۰.۵۷۸۹	۰.۵۳۶۶	۰.۵۵۷۰	۴۱
accuracy			**۰.۷۸۶۵**	**۶۷۹**
macro avg	۰.۷۷۴۷	۰.۷۶۹۰	۰.۷۷۱۳	۶۷۹
weighted avg	۰.۷۸۷۲	۰.۷۸۶۵	۰.۷۸۶۳	۶۷۹

قوی‌ترین کلاس‌ها autobus (F۱ = ۰.۹۱۷۵) و taxi (F۱ = ۰.۹۱۳۷) و ضعیف‌ترین vanet (F۱ = ۰.۵۵۷۰) هستند؛ جفت kamyun/kamyunet همچنان دشوار است.

جدول ۳۴ — ماتریس درهم‌ریختگی ثبت‌شده در نوت‌بوک آزمایش ۰۶

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۵۴	۰	۶	۲	۳	۱	۰	۷
autobus	۰	۸۱	۶	۲	۱	۲	۲	۰
kamyun	۲	۷	۶۵	۱۶	۷	۱	۰	۱
kamyunet	۴	۰	۲۲	۵۸	۵	۲	۰	۰
minibus	۱	۲	۷	۴	۶۶	۲	۰	۰
savari	۰	۱	۲	۲	۱	۸۹	۲	۳

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
taxi	۰	۲	۱	۱	۱	۵	۸۹	۰
vanet	۴	۰	۲	۵	۰	۱۴	۱	۱۵

جدول ۳۵ — مقایسه با خط مبنا (از `experiment_results.json`)

	بهترین اپوک	دقت اعتبارسنجی	خطای اعتبارسنجی در بهترین اپوک
Baseline CNN	۹	۰.۷۸۹۴	۱.۱۱۶۴
ReduceLROnPlateau	۸	۰.۷۸۶۵	۱.۰۲۰۷

تحلیل. شکاف آموزش/اعتبارسنجی نهایی ۲۱.۳۵ ٪ = ۷۸.۶۵٪ − ۱۰۰.۰٪ واحد درصد است؛ تقریباً برابر خط مبنا. زمان‌بند نرخ یادگیری دقت را بهتر نکرد (۷۸.۶۵٪ در برابر ۷۸.۹۴٪)، ولی خطای اعتبارسنجی را در نقطه ی انتخاب‌شده کمی کم کرد (۱.۰۲۱ در برابر ۱.۱۱۶). بنابراین **بیش‌برازش همچنان محدودیت اصلی است** و تنظیم نرخ یادگیری به‌تنهایی راه‌حل آن نیست.

نتیجه کلیدی: **نتیجه ی آزمایش ۰۶:** ReduceLROnPlateau دقت را بالا نبرد؛ فقط خطا را کمی بهبود داد. مشکل ریشه‌ای (ظرفیت زیاد FC و داده کم) با نرخ یادگیری حل نمی‌شود.

۳.۷ آزمایش ۰۷: دسته‌های متوازن روی ناترازی شبیه‌سازی‌شده

هدف: بررسی اینکه چگونگی ساخت دسته‌ها (Batch) چه تأثیری در حضور ناترازی کلاس‌ها دارد. چون مجموعه ی پاک‌سازی‌شده عمداً ناتراز نیست، یک ناترازی **تکرارپذیر** ساخته شد: برای سه کلاس `autobus`، `ambulance` و `kamyun` فقط **۳۰٪** نمونه‌های آموزش نگه داشته شد (بقیه دست‌نخورده). مجموعه ی آموزش از ۲,۷۱۲ به ۱,۹۶۷ نمونه رسید و اعتبارسنجی بدون تغییر ماند.

جدول ۳۶ — ناترازی شبیه‌سازی‌شده (کسر نگه‌داری ۰.۳ برای سه کلاس)

کلاس	نمونه‌های اولیه	نمونه‌های نگه‌داشته‌شده
ambulance	۲۸۹	۸۶
autobus	۳۷۶	۱۱۲
kamyun	۳۹۶	۱۱۸
kamyunet	۳۶۴	۳۶۴
minibus	۳۲۹	۳۲۹

کلاس	نمونه‌های اولیه	نمونه‌های نگه‌داشته‌شده
savari	۴۰۰	۴۰۰
taxi	۳۹۴	۳۹۴
vanet	۱۶۴	۱۶۴

دو راهبرد مقایسه شد: **(الف) دسته‌های استاندارد** با `shuffle=True` و **(ب) دسته‌های متوازن** با `BalancedBatchSampler` سفارشی.

مبنای ریاضی. در نمونه‌گیری تصادفی استاندارد احتمال دیده شدن کلاس k برابر $\pi_k = n_k/N$ است. در نمونه‌گیری متوازن به هر نمونه وزن $w_i = ۱/n_{k(i)}$ داده می‌شود تا احتمال هر کلاس در دسته $۱/K$ شود. اگر طول اپوک را همان $N = ۱۹۶۷$ فرض کنیم، ضریب «تکرار» هر نمونه در اپوک تقریباً $\frac{N/۸}{n_k}$ است:

جدول ۳۷ — ضریب تکرار هر نمونه در یک اپوک با نمونه‌گیری متوازن (محاسبه ی تقریبی با فرض طول اپوک = N)

کلاس	ambulance	autobus	kamyun	kamyunet	minibus	savari	taxi	vanet
تعداد نمونه	۸۶	۱۱۲	۱۱۸	۳۶۴	۳۲۹	۴۰۰	۳۹۴	۱۶۴
ضریب تکرار در اپوک	۲.۸۶	۲.۲۰	۲.۰۸	۰.۶۸	۰.۷۵	۰.۶۱	۰.۶۲	۱.۵۰

یعنی در نمونه‌گیری متوازن، نمونه‌های ambulance حدود ۳ بار در هر اپوک تکرار می‌شوند (حفظ کردن شدیدتر)، در حالی که نمونه‌های savari و taxi در هر اپوک فقط حدود ۶۰٪ دیده می‌شوند (تنوع داده کمتر). **این توضیح، فرض است؛ جزئیات پیاده‌سازی نمونه‌گیر در گزارش منبع نیامده است.**

اجرای الف: دسته‌های استاندارد (Shuffle)

بهترین دقت اعتبارسنجی **۷۳.۶۴٪** در اپوک ۳ است. پس از آن دقت آموزش تا حدود ۹۹.۶٪ بالا می‌رود و دقت اعتبارسنجی کم‌کم کاهش می‌یابد؛ یعنی بیش‌برازش.

جدول ۳۸ — گزارش طبقه‌بندی اجرای الف (Standard Shuffle)

کلاس	Precision	Recall	F۱
ambulance	۰.۸۲۹۸	۰.۵۴۱۷	۰.۶۵۵۵
autobus	۰.۹۵۹۵	۰.۷۵۵۳	۰.۸۴۵۲
kamyun	۰.۶۶۲۳	۰.۵۱۰۰	۰.۵۷۶۳
kamyunet	۰.۵۴۵۵	۰.۷۹۱۲	۰.۶۴۵۷
minibus	۰.۷۲۴۱	۰.۷۶۸۳	۰.۷۴۵۶

کلاس	Precision	Recall	F۱
savari	۰.۸۲۵۲	۰.۸۵۰۰	۰.۸۳۷۴
taxi	۰.۹۴۹۰	۰.۹۳۹۴	۰.۹۴۴۲
vanet	۰.۴۲۶۲	۰.۶۳۴۱	۰.۵۰۹۸

جدول ۳۹ — معیارهای کلی اجرای الف

معیار	مقدار
Accuracy	۰.۷۳۶۴
Macro Precision	۰.۷۴۰۲
Macro Recall	۰.۷۲۳۸
Macro F۱	۰.۷۲۰۰

جدول ۴۰ — ماتریس درهم‌ریختگی اجرای الف

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۳۹	۱	۳	۱۳	۲	۱	۰	۱۳
autobus	۱	۷۱	۷	۲	۵	۱	۳	۴
kamyun	۲	۱	۵۱	۲۹	۱۳	۱	۱	۲
kamyunet	۳	۰	۶	۷۲	۴	۲	۱	۳
minibus	۱	۰	۸	۸	۶۳	۱	۰	۱
savari	۱	۱	۰	۳	۰	۸۵	۰	۱۰
taxi	۰	۰	۱	۱	۰	۲	۹۳	۲
vanet	۰	۰	۱	۴	۰	۱۰	۰	۲۶

تحلیل. الگوی اصلی خطا، اشتباه کلاس‌های شبیه است: ۲۹ تصویر kamyun به‌عنوان kamyunet پیش‌بینی شده (و ۶ تصویر برعکس)؛ ۱۳ تصویر ambulance به‌عنوان kamyunet و ۱۳ تصویر به‌عنوان vanet؛ از ۴۱ نمونه ی vanet فقط ۲۶ درست و ۱۰ مورد savari پیش‌بینی شده است. در مقابل taxi با ۹۳ از ۹۹ درست عملکرد عالی دارد. توجه کنید recall کلاس‌های کم‌نمونه شده (kamyun: ۰.۵۱۰۰، ambulance: ۰.۵۴۱۷) پایین است، زیرا مدل با توزیع پیشین (Prior) ناتراز آموزش دیده ولی روی اعتبارسنجی با توزیع طبیعی سنجیده می‌شود.

اجرای ب: نمونه‌گیر متوازن (BalancedBatchSampler)

بهترین دقت اعتبارسنجی **۶۶.۸۶٪** در اپوک ۷ است؛ دقت آموزش ۹۹.۴۴٪ و بیش‌برازش دوباره وجود دارد.

جدول ۴۱ — گزارش طبقه‌بندی اجرای ب (سه ردیف آخر در خروجی اصلی نوت‌بوک بریده شده‌اند)

کلاس	Precision	Recall	F۱
ambulance	۰.۸۵۱۹	۰.۶۳۸۹	۰.۷۳۰۲
autobus	۰.۸۰۲۶	۰.۶۴۸۹	۰.۷۱۷۶
kamyun	۰.۶۳۶۴	۰.۳۵۰۰	۰.۴۵۱۶
kamyunet	۰.۵۰۹۳	۰.۶۰۴۴	۰.۵۵۲۸
minibus	۰.۶۰۰۰	۰.۸۰۴۹	۰.۶۸۷۵
savari	—	—	—
taxi	—	—	—
vanet	—	—	—

جدول ۴۲ — معیارهای کلی اجرای ب

معیار	مقدار
Accuracy	۰.۶۶۸۶
Macro Precision	۰.۶۷۰۱
Macro Recall	۰.۶۴۷۷
Macro F۱	۰.۶۴۴۸

جدول ۴۳ — ماتریس درهم‌ریختگی اجرای ب (نمونه‌گیر متوازن)

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۴۶	۰	۳	۵	۴	۸	۰	۶
autobus	۲	۶۱	۴	۱۲	۶	۵	۲	۲
kamyun	۲	۱۱	۳۵	۲۲	۲۱	۵	۰	۳
kamyunet	۱	۳	۹	۵۵	۷	۱۱	۱	۴
minibus	۱	۱	۲	۷	۶۶	۳	۰	۲
savari	۱	۰	۰	۳	۲	۸۸	۱	۵
taxi	۰	۰	۰	۰	۴	۷	۸۸	۰
vanet	۰	۰	۲	۴	۰	۲۰	۰	۱۵

تحلیل. نمونه‌گیر متوازن توزیع خطا را به‌طور محسوسی تغییر می‌دهد. Recall کلاس ambulance از ۰.۵۴۱۷ به ۰.۶۳۸۹ رسید (درست‌ها از ۳۹ به ۴۶)، ولی این بهبود به بقیه ی کلاس‌های کم‌نمونه تسری نیافت: autobus از ۷۱ به ۶۱ درست (Recall از ۰.۷۵۵۳ به ۰.۶۴۸۹) و بزرگ‌ترین افت در kamyun رخ داد (از ۵۱ به ۳۵ از ۱۰۰؛ Recall از

۰.۵۱۰۰ به ۰.۳۵۰۰). اشتباه `kamyun → kamyunet` همچنان بزرگ است (۲۲ مورد) و `vanet → savari` از ۱۰ به **۲۰** مورد افزایش یافت.

جدول ۴۴ — مقایسه ی تعداد پیش‌بینی درست در دو اجرا برای کلاس‌های کلیدی (محاسبه از ماتریس‌های درهم‌ریختگی)

کلاس	Standard درست	Balanced درست	تفاضل
ambulance	۳۹	۴۶	+۷
autobus	۷۱	۶۱	−۱۰
kamyun	۵۱	۳۵	−۱۶
vanet	۲۶	۱۵	−۱۱

چرا نمونه‌گیر متوازن بدتر شد؟ سه عامل محتمل: **(۱)** تکرار زیاد نمونه‌های معدود ⟵ حفظ بیشتر و بیش‌برازش؛ **(۲)** کم دیده شدن نمونه‌های کلاس‌های پرتعداد ⟵ کاهش تنوع و افت دقت آن‌ها؛ **(۳)** جابه‌جایی توزیع پیشین. اگر مدل با پیشین یکنواخت $1/K$ آموزش ببیند ولی روی داده ی طبیعی با پیشین π_k سنجیده شود، پسین‌ها باید اصلاح شوند:

$$p_{\text{test}}(k \mid x) \;\propto\; p_{\text{bal}}(k \mid x) \cdot \frac{\pi_k^{test}}{1/K}$$

بدون چنین اصلاحی (مثلاً اصلاح لوجیت) مرزهای تصمیم برای کلاس‌های پرتعداد جابه‌جا می‌شوند.

نتیجه کلیدی: **نتیجه ی آزمایش ۰۷:** در این ناترازی (نسبت ۳۰٪)، نمونه‌گیر متوازن دقت کلی را از ۷۳.۶۴٪ به ۶۶.۸۶٪ کاهش داد. Recall کلاس ambulance بهتر شد، اما برای autobus، kamyun و vanet بدتر شد. «متوازن‌سازی دسته‌ها» بدون اصلاحات دیگر (وزن‌دهی خطا، افزایش داده برای کلاس‌های کم‌نمونه، اصلاح پیشین) تضمین بهبود نیست.

۳.۸ آزمایش ۰۸: CrossEntropy در برابر BCE

هدف: مقایسه ی دو تابع هزینه روی همین مسئله ی ۸ کلاسه. تنها تابع هزینه و نمایش هدف تغییر می‌کند. هدف CE یک شاخص صحیح کلاس است، و هدف BCE یک بردار یک‌هات $y \in \{۰,۱\}^۸$ است. تشخیص کلاس نهایی در هر دو با argmax روی لوجیت‌ها انجام می‌شود.

مبنای ریاضی.

$$\mathcal{L}_{CE} = -\log\frac{e^{z_y}}{\sum_j e^{z_j}}, \qquad \mathcal{L}_{BCE} = -\sum_{k=1}^{K}\left[y_k \log\sigma(z_k) + (1-y_k)\log\left(1-\sigma(z_k)\right)\right]$$

در CE، Softmax کلاس‌ها را به **رقابت** وامی‌دارد: افزایش احتمال یک کلاس، احتمال بقیه را کم می‌کند و مجموع احتمال‌ها ۱ است. در BCE هر خروجی یک مسئله ی دودویی **مستقل** ($\sigma(z) = 1/(1+e^{-z})$) است و مجموع خروجی‌ها لزوماً ۱ نیست. گرادیان هر دو نسبت به لوجیت فرم مشابه «پیش‌بینی منهای هدف» دارد ($p_k - y_k$ یا $\sigma(z_k) - y_k$)، ولی در BCE هیچ بهنجارسازی بین کلاس‌ها وجود ندارد. بنابراین BCE برای مسئله‌های **چندبرچسبی** طبیعی‌تر است و CE برای مسئله ی **تک‌برچسبیِ** ما (هر تصویر دقیقاً یک کلاس).

اجرای الف: CrossEntropy

۵۴۶ از ۶۷۹ پیش‌بینی درست؛ دقت اعتبارسنجی **۸۰.۴۱٪**، Macro Recall = ۷۹.۰۳٪ و Macro Precision = ۷۹.۵۱٪، Macro F۱ = ۷۹.۱۷٪.

جدول ۴۵ — گزارش طبقه‌بندی CE

کلاس	Precision	Recall	F۱
ambulance	۰.۸۶۰۸	۰.۹۴۴۴	۰.۹۰۰۷
autobus	۰.۹۰۶۲	۰.۹۲۵۵	۰.۹۱۵۸
kamyun	۰.۷۰۰۰	۰.۷۰۰۰	۰.۷۰۰۰
kamyunet	۰.۶۳۱۶	۰.۶۵۹۳	۰.۶۴۵۲
minibus	۰.۸۵۵۳	۰.۷۹۲۷	۰.۸۲۲۸
savari	۰.۸۲۱۸	۰.۸۳۰۰	۰.۸۲۵۹
taxi	۰.۹۲۷۸	۰.۹۰۹۱	۰.۹۱۸۴
vanet	۰.۶۵۷۱	۰.۵۶۱۰	۰.۶۰۵۳

جدول ۴۶ — ماتریس درهم‌ریختگی CE

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۶۸	۰	۱	۲	۰	۰	۰	۱
autobus	۰	۸۷	۵	۱	۱	۰	۰	۰
kamyun	۲	۶	۷۰	۱۵	۳	۲	۱	۱
kamyunet	۶	۱	۱۳	۶۰	۶	۳	۰	۲
minibus	۰	۰	۶	۶	۶۵	۱	۴	۰
savari	۱	۲	۱	۵	۰	۸۳	۱	۷

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
taxi	۰	۰	۲	۲	۱	۳	۹۰	۱
vanet	۲	۰	۲	۴	۰	۹	۱	۲۳

بزرگ‌ترین منبع خطا باز هم جفت kamyun/kamyunet است: از ۱۰۰ نمونه ی ۷۰ ،kamyun درست و ۱۵ مورد kamyunet؛ از ۹۱ نمونه ی ۶۰ ،kamyunet درست و ۱۳ مورد kamyun. نقطه ی ضعف دیگر vanet است: فقط ۲۳ از ۴۱ درست، ۹ مورد savari و ۴ مورد kamyunet. در مقابل ambulance (۶۸ از ۷۲) و autobus (۸۷ از ۹۴) خیلی خوب شناسایی می‌شوند.

اجرای ب: BCEWithLogitsLoss

۵۳۵ از ۶۷۹ پیش‌بینی درست؛ دقت **۷۸.۷۹٪**، Macro Recall = ۷۷.۰۴٪، Macro Precision = ۷۸.۲۵٪ و Macro F۱ = ۷۷.۱۸٪.

جدول ۴۷ — گزارش طبقه‌بندی BCE (سه ردیف آخر در خروجی اصلی نوت‌بوک بریده شده‌اند)

کلاس	Precision	Recall	F۱
ambulance	۰.۸۱۸۲	۰.۸۷۵۰	۰.۸۴۵۶
autobus	۰.۸۶۶۰	۰.۸۹۳۶	۰.۸۷۹۶
kamyun	۰.۶۵۰۰	۰.۵۲۰۰	۰.۵۷۷۸
kamyunet	۰.۶۱۶۱	۰.۷۵۸۲	۰.۶۷۹۸
minibus	۰.۸۲۹۳	۰.۸۲۹۳	۰.۸۲۹۳
savari	—	—	—
taxi	—	—	—
vanet	—	—	—

جدول ۴۸ — ماتریس درهم‌ریختگی BCE

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۶۳	۰	۰	۴	۰	۳	۰	۲
autobus	۱	۸۵	۴	۱	۳	۰	۱	۰
kamyun	۳	۱۰	۵۲	۲۵	۷	۲	۰	۱
kamyunet	۵	۰	۱۰	۶۹	۳	۲	۲	۰
minibus	۰	۱	۸	۵	۶۸	۰	۰	۰
savari	۱	۱	۳	۲	۰	۸۶	۳	۴
taxi	۰	۰	۱	۱	۱	۲	۹۳	۱
vanet	۴	۱	۲	۵	۰	۹	۰	۲۰

الگوی خطا مشابه است ولی شدتش فرق می‌کند: فقط ۵۲ از ۱۰۰ نمونه ی kamyun درست (در برابر ۷۰ در CE)، با ۲۵ مورد kamyunet → kamyun و ۱۰ مورد kamyun → kamyunet. برای vanet: ۲۰ از ۴۱ درست (در برابر ۲۳ در CE)، و ۹ مورد savari در هر دو مدل مشترک است.

مقایسه ی مستقیم

جدول ۴۹ — مقایسه ی تعداد پیش‌بینی درست به‌تفکیک کلاس

کلاس واقعی	درست در CE	درست در BCE	تفاضل
ambulance	۶۸	۶۳	CE +۵
autobus	۸۷	۸۵	CE +۲
kamyun	**۷۰**	**۵۲**	**CE +۱۸**
kamyunet	۶۰	۶۹	CE −۹
minibus	۶۵	۶۸	CE −۳
savari	۸۳	۸۶	CE −۳
taxi	۹۰	۹۳	CE −۳
vanet	۲۳	۲۰	CE +۳

CE در **همه ی** کلاس‌ها بهتر نیست: BCE برای kamyunet (۶۹ در برابر ۶۰)، minibus (۶۸ در برابر ۶۵)، savari (۸۶ در برابر ۸۳) و taxi (۹۳ در برابر ۹۰) درست‌های بیشتری دارد. اما CE در kamyun با اختلاف **۱۸ نمونه** (۷۰ در برابر ۵۲) پیشی می‌گیرد؛ پس برتری کلی CE از عملکرد بهتر در برخی کلاس‌های دشوار—به‌ویژه kamyun—ناشی می‌شود نه برتری همگانی.

جدول ۵۰ — مقایسه ی کمّی نهایی CE و BCE

معیار	CE	BCE	تفاضل
Accuracy	۰.۸۰۴۱	۰.۷۸۷۹	+۱.۶۲ pp
Macro Precision	۰.۷۹۵۱	۰.۷۸۲۵	+۱.۲۶ pp
Macro Recall	۰.۷۹۰۳	۰.۷۷۰۴	+۱.۹۸ pp
Macro F۱	۰.۷۹۱۷	۰.۷۷۱۸	+۱.۹۹ pp

CE در **همه ی** معیارهای تجمیعی بهتر است؛ بیشترین اختلاف در Macro-F۱ است:

۰.۰۱۹۹ = ۰.۷۷۱۸ − ۰.۷۹۱۷ (≈ ۱.۹۹ واحد درصد).

رفتار اطمینان (Confidence) و خطاهای با اطمینان بالا

CE با Softmax یک توزیع احتمال می‌دهد که مجموعش ۱ است؛ BCE با Sigmoidهای مستقل خروجی می‌دهد که مجموعشان ۱ نیست. مثال از خروجی نوت‌بوک:

```
CE  : [0.000004, 0.002646, 0.021352, 0.963950, 0.011806, 0.000016, 0.000093,
                                                                    [0.000133
BCE : [0.000150, 0.025038, 0.332890, 0.857250, 0.002555, 0.000480, 0.001040,
                                                                    [0.001731
```

بنابراین عدد ۰.۸۵۷ در BCE را نباید دقیقاً مثل احتمال Softmax برابر ۰.۸۵۷ تفسیر کرد؛ این موضوع برای آستانه‌گذاری اطمینان در مرحله ی `needs_review` و تشخیص کلاس ناشناخته مهم است. همچنین خروجی نوت‌بوک نشان می‌دهد پیش‌بینی‌های اشتباه با اطمینان بسیار بالا وجود دارند؛ مثلاً (= True minibus، Pred = kamyun، اطمینان ۰.۹۵۳۹) و (True = kamyun، Pred = taxi، اطمینان ۰.۹۸۸۵). پس اطمینان Softmax به‌تنهایی صحت را تضمین نمی‌کند—موضوعی که در آزمایش ۱۴ به‌صورت جدی بررسی می‌شود.

> نتیجه کلیدی: **نتیجه ی آزمایش ۰۸:** CE برای این مسئله ی تک‌برچسبی بهتر است (۸۰.۴۱٪ در برابر ۷۸.۷۹٪)، فرمول‌بندی‌اش با کلاس‌های ناسازگار هم‌خوان است و احتمال‌های بهنجارشده می‌دهد. **CE برای مدل نهایی انتخاب شد.**

۳.۹ آزمایش ۰۹: Global Average Pooling (GAP)

هدف: آیا کاهش شدید ظرفیت لایه ی FC با GAP بیش‌برازش را کم می‌کند؟ به‌جای Flatten کردن همه ی نقشه‌های ویژگی (۵۶ × ۵۶ × ۶۴)، میانگین هر نقشه یک عدد می‌شود:

$$g_c = \frac{1}{H\,W}\sum_{i=1}^{H}\sum_{j=1}^{W} F_c\,(i,j), \qquad c = 1,\ldots,64$$

```
بـدون GAP:  64 x 56 x 56  ->  Flatten  ->  200,704  ->  128  ->  8
بـا GAP:    64 x 56 x 56  ->  GAP      ->  64       ->  128  ->  8
```

تعداد پارامترهای FC۱ از ۲۵,۶۹۰,۲۴۰ = ۱۲۸ + ۱۲۸ × ۲۰۰۷۰۴ به ۸,۳۲۰ = ۱۲۸ + ۱۲۸ × ۶۴ می‌رسد؛ یعنی حدود **۳۰۸۸ برابر** کمتر. مدل حداکثر ۵۰ اپوک با Early Stopping آموزش دید.

جدول ۵۱ — لاگ آموزش آزمایش ۰۹ (GAP). اپوک‌های میانی در خروجی منبع نیامده‌اند.

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی	نرخ یادگیری
۱	۱.۹۸۲۸	۰.۲۲۹۰	۱.۸۵۱۲	۰.۳۳۱۴	۰.۰۰۱۰۰۰
۲	۱.۸۱۴۴	۰.۳۰۲۷	۱.۷۸۷۴	۰.۳۴۱۷	۰.۰۰۱۰۰۰
۳	۱.۷۳۰۴	۰.۳۴۸۱	۱.۶۸۴۹	۰.۳۶۵۲	۰.۰۰۱۰۰۰
۴	۱.۶۵۸۴	۰.۳۸۰۹	۱.۵۹۲۱	۰.۴۲۱۲	۰.۰۰۱۰۰۰
۵	۱.۵۹۷۳	۰.۴۰۲۷	۱.۵۶۰۳	۰.۴۲۲۷	۰.۰۰۱۰۰۰
۶	۱.۵۷۳۸	۰.۳۹۴۵	۱.۵۳۴۳	۰.۴۳۴۵	۰.۰۰۱۰۰۰
۷	۱.۵۲۹۳	۰.۴۲۵۵	۱.۴۹۱۱	۰.۴۴۶۲	۰.۰۰۱۰۰۰
۸	۱.۴۹۷۹	۰.۴۳۳۳	۱.۴۵۱۲	۰.۴۷۵۷	۰.۰۰۱۰۰۰
۹	۱.۴۵۳۳	۰.۴۶۰۲	۱.۴۱۹۹	۰.۴۶۹۸	۰.۰۰۱۰۰۰
۱۰	۱.۴۲۴۴	۰.۴۷۰۱	۱.۳۸۹۳	۰.۴۸۳۱	۰.۰۰۱۰۰۰
۱۱	۱.۳۷۸۰	۰.۴۸۱۹	۱.۳۳۶۶	۰.۵۰۲۲	۰.۰۰۱۰۰۰
۱۲	۱.۳۵۶۲	۰.۴۸۸۹	۱.۳۹۸۰	۰.۴۷۴۲	۰.۰۰۱۰۰۰
۱۳	۱.۳۳۴۸	۰.۵۱۵۹	۱.۲۹۹۹	۰.۵۳۴۶	۰.۰۰۱۰۰۰
…	…	…	…	…	…
۲۲	—	—	—	۰.۵۸۴۷	۰.۰۰۱۰۰۰
۲۵	—	—	—	—	۰.۰۰۱۰۰۰

Early Stopping در اپوک ۲۵ فعال شد. بهترین دقت اعتبارسنجی **۵۸.۴۷٪** (۳۹۷ از ۶۷۹) در اپوک ۲۲ است.

جدول ۵۲ — معیارهای تجمیعی GAP

معیار	GAP
Accuracy	۵۸.۴۷٪
Macro Precision	۵۲.۸۱٪
Macro Recall	۵۳.۸۷٪
Macro F۱	۵۲.۶۹٪

جدول ۵۳ — گزارش طبقه‌بندی آزمایش ۰۹ (GAP)

کلاس	Precision	Recall	F۱	Support
ambulance	۰.۶۰۶۱	۰.۵۵۵۶	۰.۵۷۹۷	۷۲
autobus	۰.۵۰۴۹	۰.۵۵۳۲	۰.۵۲۷۹	۹۴
kamyun	۰.۶۹۴۴	۰.۵۰۰۰	۰.۵۸۱۴	۱۰۰
kamyunet	۰.۳۴۴۸	۰.۵۴۹۵	۰.۴۲۳۷	۹۱
minibus	۰.۶۱۷۶	۰.۵۱۲۲	۰.۵۶۰۰	۸۲

کلاس	Precision	Recall	F۱	Support
savari	۰.۶۳۷۲	۰.۷۲۰۰	۰.۶۷۶۱	۱۰۰
taxi	۰.۸۱۹۸	۰.۹۱۹۲	۰.۸۶۶۷	۹۹
vanet	۰.۰۰۰۰	۰.۰۰۰۰	۰.۰۰۰۰	۴۱
Accuracy			**۰.۵۸۴۷**	**۶۷۹**
Macro Avg	۰.۵۲۸۱	۰.۵۳۸۷	۰.۵۲۶۹	۶۷۹
Weighted Avg	۰.۵۷۰۶	۰.۵۸۴۷	۰.۵۷۰۵	۶۷۹

جدول ۵۴ — ماتریس درهم‌ریختگی آزمایش ۰۹ (GAP)

True \ Pred	amb	auto	kamyun	kamyunet	mini	savari	taxi	vanet
ambulance	۴۰	۴	۰	۲۰	۳۰	۸	۰	۰
autobus	۵	۵۲	۳	۱۸	۷	۲	۷	۰
kamyun	۰	۱۵	۵۰	۲۵	۳	۲	۴	۱
kamyunet	۶	۸	۸	۵۰	۱۳	۳	۳	۰
minibus	۰	۱۱	۸	۱۵	۴۲	۴	۲	۰
savari	۹	۸	۰	۷	۱	۷۲	۳	۰
taxi	۱	۳	۰	۲	۱	۱	۹۱	۰
vanet	۵	۲	۳	۸	۱	۲۱	۱	۰

تحلیل به‌تفکیک کلاس

جدول ۵۵ — تحلیل کلاس‌های برجسته ی GAP

کلاس	Precision	Recall	F۱	توضیح
taxi	۰.۸۱۹۸	۰.۹۱۹۲	۰.۸۶۶۷	بهترین؛ اکثر تاکسی‌ها درست شناسایی می‌شوند
savari	۰.۶۳۷۲	۰.۷۲۰۰	۰.۶۷۶۱	دومین مقدار بالا Recall
kamyun	۰.۶۹۴۴	۰.۵۰۰۰	۰.۵۸۱۴	Precision نسبتاً بالا ولی نیمی از نمونه‌ها از دست می‌روند
kamyunet	۰.۳۴۴۸	۰.۵۴۹۵	۰.۴۲۳۷	Precision بسیار کم؛ بسیاری از پیش‌بینی‌های kamyunet نادرست‌اند
vanet	۰.۰۰۰۰	۰.۰۰۰۰	۰.۰۰۰۰	هیچ‌یک از ۴۱ نمونه درست شناسایی نشد

چرا GAP در اینجا شکست خورد؟

(الف) از دست رفتن اطلاعات مکانی. بدون GAP لایه ی FC هم «کدام ویژگی» و هم «کجا بودن آن» را می‌بیند؛ GAP فقط میانگین فعال‌سازی هر نقشه را نگه می‌دارد و موقعیت را کنار می‌گذارد. برای تفکیک وسایل نقلیه ی مشابه، چیدمان اجزا (پنجره‌ها، چرخ‌ها، چراغ‌ها، بدنه) مهم است.

(ب) میدان دید مؤثر کوچک. میدان دید (Receptive Field) هر سلول خروجی این CNN کوچک بسیار محدود است: Conv۱ با هسته ی ۳: ۳ پیکسل؛ Pool۱: ۴؛ Conv۲: ۸ = ۲ × ۲ + ۴؛ Pool۲: ۱۰ پیکسل. بنابراین هر سلول نقشه ی ۵۶ × ۵۶ فقط یک تکه ی حدود ۱۰ × ۱۰ از تصویر ۲۲۴ × ۲۲۴ را می‌بیند و ویژگی‌هایش «محلی/بافتی» است. میانگین‌گیری سراسری روی چنین ویژگی‌های محلی، شکل کلی خودرو را نمی‌سازد؛ برخلاف ResNet عمیق که پیش از GAP میدان دید بسیار بزرگ دارد.

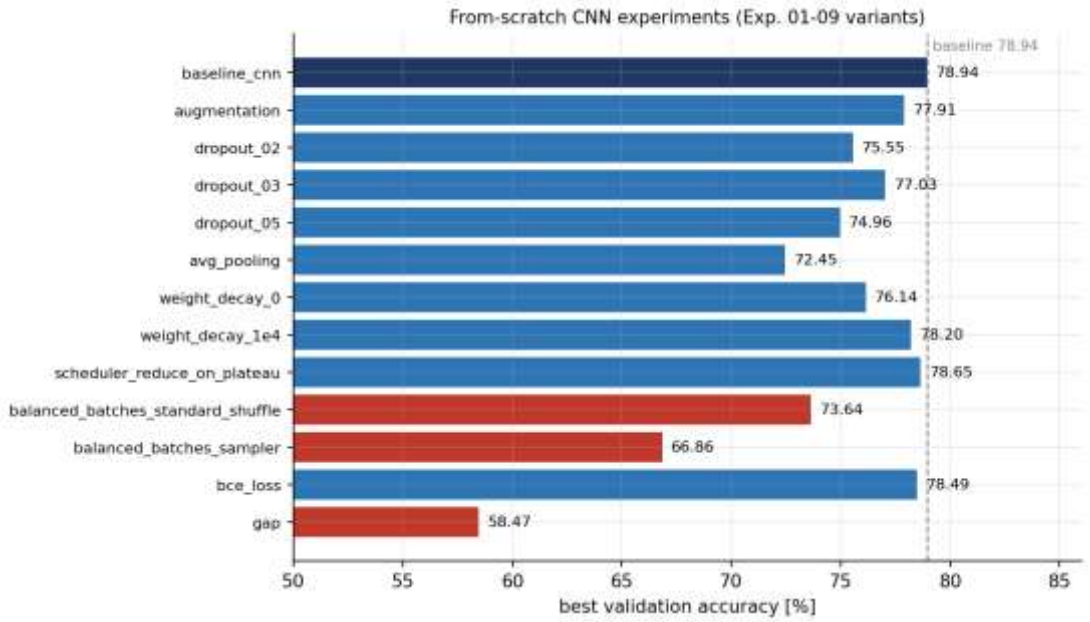
(ج) کم‌برازش یا همگرایی کند. دقت آموزش در اپوک ۱۳ فقط حدود ۵۱.۶٪ است و در طول اجرا همچنان آهسته بالا می‌رفت. این نشان می‌دهد مدل نه تنها بیش‌برازش ندارد، بلکه کم‌برازش دارد. این نتیجه با فرضیه ی اولیه (کاهش ظرفیت برای کاهش بیش‌برازش) هم‌خوان نیست: ظرفیت بسیار کم، مدل را به «کم‌برازش» برد. شاید ادامه ی آموزش با Early Stopping آهسته‌تر نیز کمی بهتر می‌شد، اما با ۵۸.۴۷٪ فاصله با دیگر آزمایش‌ها بسیار زیاد است.

جدول ۵۶ — مقایسه ی GAP با آزمایش‌های پیشین

آزمایش	دقت اعتبارسنجی
GAP	**۵۸.۴۷٪**
Balanced Batches	۷۳.۶۴٪
Scheduler	۷۸.۶۵٪
CrossEntropy	۸۰.۴۱٪

نتیجه کلیدی: **نتیجه ی آزمایش ۰۹:** GAP برای این معماری کوچک مناسب نیست (۵۸.۴۷٪). کاهش پارامتر لزوماً تعمیم بهتر نمی‌دهد: منظم‌سازی باید ظرفیت لازم برای یادگیری را حفظ کند. این نتیجه به همین معماری و مجموعه‌داده محدود است و درباره ی GAP در شبکه‌های عمیق‌تر (مانند ResNet که GAP دارد) صدق نمی‌کند.

۳.۱۰ جمع‌بندی فاز اول



شکل ۶ — بهترین دقت اعتبارسنجی هر پیکربندی در فاز اول (داده از جدول Result.json).

فاز اول چند درس روشن می‌دهد:

- **سقف CNN ساده ≈ ۷۸–۸۰٪.** هیچ‌یک از ترفندها (Dropout، افزایش داده، weight decay، زمان‌بند) به‌طور معنادار از خط مبنا (۷۸.۹۴٪) بهتر نشد؛ بهترین دقت ۸۰.۴۱٪ در اجرای CE (آزمایش ۰۸) به‌دست آمد.
- **بیش‌برازش ریشه‌ای است.** ساختار FC با ۲۵.۷ میلیون پارامتر و داده ی کم، شکاف ۲۰+ واحد درصدی ایجاد می‌کند و منظم‌سازهای سبک آن را حل نمی‌کنند.
- **نوسان تصادفی بزرگ است.** تفاوت بین اجراها (مثلاً AdamW با $\lambda = 0$ در برابر Adam: ≈ ۲.۸ واحد) هم‌اندازه یا بزرگ‌تر از اثر بیشتر ترفندهاست؛ پس تفاضل‌های ۱–۲ واحدی باید محتاطانه تفسیر شوند.
- **خطاهای ساختاری پایدار.** در تمام آزمایش‌ها `kamyun ↔ kamyunet` و `vanet → savari` بزرگ‌ترین خطاها هستند؛ یعنی ظرفیت بازنمایی مدل (نه جزئیات آموزش) محدودکننده است.
- **پیامد برای طراحی:** برای عبور از این سقف باید بازنمایی ویژگی‌ها قوی‌تر شود؛ **یادگیری انتقالی** (فاز دوم) پاسخ طبیعی است.

۴. فاز دوم: یادگیری انتقالی با ResNet۱۸ (آزمایش‌های ۱۰ و ۱۱)

۴.۱ چرا یادگیری انتقالی؟ مبانی نظری

درس فاز اول این بود که یک CNN کوچک از صفر با ۳ هزار تصویر نمی‌تواند بازنمایی کافی بسازد. **یادگیری انتقالی** (**Transfer Learning**) از شبکه‌ای استفاده می‌کند که روی ImageNet (حدود ۱.۲۸ میلیون تصویر، ۱۰۰۰ کلاس) آموزش دیده و لبه‌ها، بافت‌ها، اجزا و اشیای عمومی را از قبل می‌شناسد. لایه‌های اولیه ویژگی‌های عمومی (لبه، رنگ) و لایه‌های عمیق‌تر ویژگی‌های اختصاصی‌تر (اجزای شیء) را استخراج می‌کنند؛ پس انتقال‌پذیری لایه‌های اول بالاست.

الف) بلوک باقیمانده (Residual) در ResNet

ResNet۱۸ از بلوک‌هایی ساخته شده که به‌جای یادگیری مستقیم نگاشت $H(x)$، تابع «باقیمانده» $F(x) = H(x) - x$ را یاد می‌گیرند و ورودی را از راه میان‌بُر به خروجی اضافه می‌کنند:

$$y = \mathcal{F}(x, \{W_i\}) + x, \qquad \mathcal{F}(x) = W_2\,\sigma\big(\mathrm{BN}(W_1 x)\big)$$

در پس‌انتشار، $\partial y/\partial x = \partial \mathcal{F}/\partial x + I$ است و جمله ی همانی I تضمین می‌کند گرادیان حتی در شبکه‌های عمیق از بین نرود (مسئله ی محوشدگی گرادیان). همچنین **Batch Normalization** هر کانال را در دسته نرمال می‌کند: $\hat{x} = (x - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$ و سپس $y = \gamma \hat{x} + \beta$.

ساختار ResNet۱۸: لایه ی اولیه (`conv1` و `maxpool`)، چهار مرحله `layer1` تا `layer4` هر کدام با ۲ بلوک، سپس **Global Average Pooling** و یک لایه ی تماماً متصل. لایه ی ۱۰۰۰ کلاسه ی آخر با لایه ی ۸ کلاسه جایگزین می‌شود: $512 \times 8 + 8 = 4104$ پارامتر. توجه کنید: در اینجا GAP **مشکلی ندارد**، زیرا پیش از آن ۱۷ لایه ی کانولوشن میدان دید بسیار بزرگی ساخته است (برخلاف آزمایش ۰۹).

ب) سه راهبرد تنظیم

جدول ۵۷ — سه راهبرد یادگیری انتقالی (نرخ‌های یادگیری از Result.json)

راهبرد	لایه‌های قابل آموزش	نرخ یادگیری	پارامترهای قابل آموزش
Feature Extraction	فقط fc جدید	10^{-3}	۴,۱۰۴
Partial Fine-Tuning	layer۴ + fc	layer۴: 10^{-4} ، fc: 10^{-3}	۸,۳۹۷,۸۳۲
Full Fine-Tuning	همه ی لایه‌ها	backbone: 10^{-4} ، head: 10^{-3}	۱۱,۱۸۰,۶۱۶

در Feature Extraction مدل عملاً یک رگرسیون لجستیک خطی روی ویژگی‌های ثابت ۵۱۲ بعدی است: $z = W\,\phi(x) + b$ با ϕ ثابت. در Fine-Tuning علاوه بر W پارامترهای ϕ هم بهینه می‌شوند. استفاده از **نرخ یادگیری متفاوت** (کوچک‌تر برای backbone) این منطق را دارد که وزن‌های پیش‌آموزش‌دیده باید فقط اندکی جابه‌جا شوند ولی لایه ی تازه‌مقداردهی‌شده ی fc باید سریع یاد بگیرد:

$$\theta_{backbone} \leftarrow \theta_{backbone} - \eta_b \nabla, \quad \theta_{head} \leftarrow \theta_{head} - \eta_h \nabla, \qquad \eta_b = ۱۰^{-۴} \ll \eta_h = ۱۰^{-۳}$$

۴.۲ آزمایش ۱۰: نتایج سه راهبرد

تصاویر ۲۲۴×۲۲۴ هستند و در آزمایش‌های اصلی نرمال‌سازی ImageNet اعمال می‌شود. مدل‌گزینی بر پایه ی اعتبارسنجی است و مجموعه ی آزمون در انتخاب استفاده نمی‌شود.

الف) Feature Extraction

جدول ۵۸ — لاگ آموزش Feature Extraction ResNet۱۸ (Early Stopping در اپوک ۱۶)

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۱.۴۶۸۹	۰.۵۱۹۵	۰.۹۷۴۶	۰.۷۲۱۶
۲	۰.۸۵۳۱	۰.۷۶۰۷	۰.۷۲۶۲	۰.۷۹۶۸
۳	۰.۶۸۰۶	۰.۸۱۱۹	۰.۶۷۰۳	۰.۷۹۹۷
۴	۰.۵۸۲۹	۰.۸۳۵۵	۰.۵۶۰۸	۰.۸۳۲۱
۵	۰.۵۳۰۰	۰.۸۴۶۶	۰.۵۳۰۱	۰.۸۲۹۲
۶	۰.۴۸۳۸	۰.۸۶۲۸	۰.۴۹۱۵	۰.۸۵۲۷
۷	۰.۴۵۸۴	۰.۸۶۱۷	۰.۴۸۶۱	۰.۸۵۴۲
۸	۰.۴۳۷۵	۰.۸۶۳۲	۰.۴۵۶۳	۰.۸۵۲۷
۹	۰.۴۰۲۳	۰.۸۷۰۲	۰.۴۴۲۸	۰.۸۵۵۷
۱۰	۰.۳۹۳۹	۰.۸۸۰۹	۰.۴۶۶۰	۰.۸۳۶۵
۱۱	**۰.۳۷۹۹**	**۰.۸۸۶۸**	**۰.۴۲۰۳**	**۰.۸۶۸۹**
۱۲	۰.۳۴۳۹	۰.۹۰۴۱	۰.۴۲۶۴	۰.۸۶۰۱
۱۳	۰.۳۳۹۳	۰.۸۹۵۶	۰.۴۲۱۴	۰.۸۶۴۵
۱۴	۰.۳۳۸۷	۰.۸۹۶۴	۰.۴۱۷۰	۰.۸۶۸۹
۱۵	۰.۳۱۶۳	۰.۹۰۰۱	۰.۴۱۳۶	۰.۸۶۸۹
۱۶	۰.۳۱۰۴	۰.۹۱۰۴	۰.۴۲۳۵	۰.۸۶۳۰

بهترین دقت **۸۶.۸۹٪** در اپوک ۱۱ است. معیارها: Accuracy = ۸۶.۸۹٪، Macro Precision = ۸۶.۱۸٪، Weighted F۱ = ۸۶.۷۹٪، Macro F۱ = ۸۵.۹۱٪، Macro Recall = ۸۵.۸۷٪. این نتیجه نشان می‌دهد ویژگی‌های ImageNet بدون هیچ تنظیمی و فقط با آموزش ۴,۱۰۴ پارامتر، ۸ واحد درصد از CNN آموزش‌دیده از صفر بهترند.

ب) Partial Fine—Tuning (layer۴ + fc)

جدول ۵۹ — لاگ آموزش ResNet۱۸ با باز کردن layer۴ (Early Stopping در اپوک ۱۷)

اپوک	خطای آموزش	دقت آموزش	خطای اعتبارسنجی	دقت اعتبارسنجی
۱	۰.۳۱۲۷	۰.۸۹۵۶	۰.۲۹۰۱	۰.۸۹۹۹
۲	۰.۰۴۱۹	۰.۹۹۱۹	۰.۲۶۲۴	۰.۹۲۴۹
۳	۰.۰۱۱۰	۰.۹۹۹۳	۰.۲۵۰۲	۰.۹۲۷۸
۴	۰.۰۰۵۳	۱.۰۰۰۰	۰.۲۴۷۳	۰.۹۳۰۸
۵	۰.۰۰۳۰	۱.۰۰۰۰	۰.۲۴۳۹	۰.۹۲۹۳
۶	۰.۰۰۲۸	۱.۰۰۰۰	۰.۲۵۲۸	۰.۹۳۳۷
۷	۰.۰۰۱۷	۱.۰۰۰۰	۰.۲۵۶۸	۰.۹۳۲۳
۸	۰.۰۰۱۱	۱.۰۰۰۰	۰.۲۶۰۲	۰.۹۳۵۲
۹	۰.۰۰۰۹	۱.۰۰۰۰	۰.۲۵۵۱	۰.۹۳۵۲
۱۰	۰.۰۰۰۶	۱.۰۰۰۰	۰.۲۶۶۷	۰.۹۳۶۷
۱۱	۰.۰۰۰۷	۱.۰۰۰۰	۰.۲۶۷۹	۰.۹۳۳۷
۱۲	**۰.۰۰۰۷**	**۱.۰۰۰۰**	**۰.۲۷۰۳**	**۰.۹۳۹۶**
۱۳	۰.۰۱۸۷	۰.۹۹۵۶	۰.۴۶۸۱	۰.۹۰۵۷
۱۴	۰.۰۲۳۸	۰.۹۹۳۴	۰.۳۲۹۵	۰.۹۲۷۸
۱۵	۰.۰۱۲۷	۰.۹۹۵۶	۰.۴۰۴۸	۰.۹۳۶۷
۱۶	۰.۰۱۵۵	۰.۹۹۵۲	۰.۳۵۸۲	۰.۹۲۳۴
۱۷	۰.۰۲۹۷	۰.۹۹۱۲	۰.۷۳۷۸	۰.۹۰۱۳

بهترین دقت **۹۳.۹۶٪** در اپوک ۱۲ است؛ بهبود ۷.۰۷ = ۸۶.۸۹ − ۹۳.۹۶ واحد درصد. دقت آموزش در اپوک ۴ به ۱۰۰٪ می‌رسد. از اپوک ۱۳ به بعد تنظیم ناپایدار می‌شود: خطای آموزش دوباره بالا می‌رود (۰.۰۱۸۷) و خطای اعتبارسنجی به ۰.۴۶۸۱ و سپس ۰.۷۳۷۸ می‌رسد. این ناپایداری (احتمالاً ناشی از نرخ یادگیری 10^{-3} برای head) توجیه‌کننده ی Early Stopping و انتخاب بهترین اپوک است.

پ) Full Fine—Tuning

جدول ۶۰ — مقایسه ی سه راهبرد

راهبرد	دقت اعتبارسنجی	Macro F۱	پارامترهای قابل آموزش
Feature Extraction	۸۶.۸۹٪	۸۵.۹۱٪	۴,۱۰۴
Layer۴ + FC Fine-Tuning	۹۳.۹۶٪	۹۳.۷۷٪	۸,۳۹۷,۸۳۲
Full Fine-Tuning	**۹۶.۰۲٪**	**۹۵.۷۳٪**	**۱۱,۱۸۰,۶۱۶**

جدول ۶۱ — گزارش طبقه‌بندی Full Fine-Tuning (اعتبارسنجی)

کلاس	Precision	Recall	F۱
ambulance	۱.۰۰	۰.۹۹	۰.۹۹
autobus	۰.۹۹	۰.۹۹	۰.۹۹
kamyun	۰.۹۴	۰.۹۲	۰.۹۳
kamyunet	۰.۹۰	۰.۹۷	۰.۹۳
minibus	۰.۹۶	۰.۹۶	۰.۹۶
savari	۰.۹۷	۰.۹۶	۰.۹۶
taxi	۱.۰۰	۰.۹۷	۰.۹۸
vanet	۰.۹۰	۰.۹۰	۰.۹۰

معیارهای کلی: Macro Recall = ۹۵.۷۳٪، Macro Precision = ۹۵.۷۷٪، Accuracy = ۹۶.۰۲٪، Weighted F۱ = ۹۶.۰۴٪، F۱ = ۹۵.۷۳٪. تقریباً همه ی کلاس‌ها بالای ۰.۹۳ هستند؛ ضعیف‌ترین کلاس vanet با F۱ = ۰.۹۰ است (در CNN پایه ۰.۵۸ بود).

بهبود به تفکیک مرحله: layer۴ → FE: +۷.۰۷؛ Full → layer۴: +۲.۰۶؛ Full → FE: +۹.۱۳ واحد درصد.

ت) اثر نرمال‌سازی ImageNet

جدول ۶۲ — Full Fine-Tuning با و بدون نرمال‌سازی ImageNet

تنظیم Full Fine-Tuning	بهترین دقت اعتبارسنجی
With ImageNet normalization	۹۶.۰۲٪
Without normalization	۹۶.۳۲٪

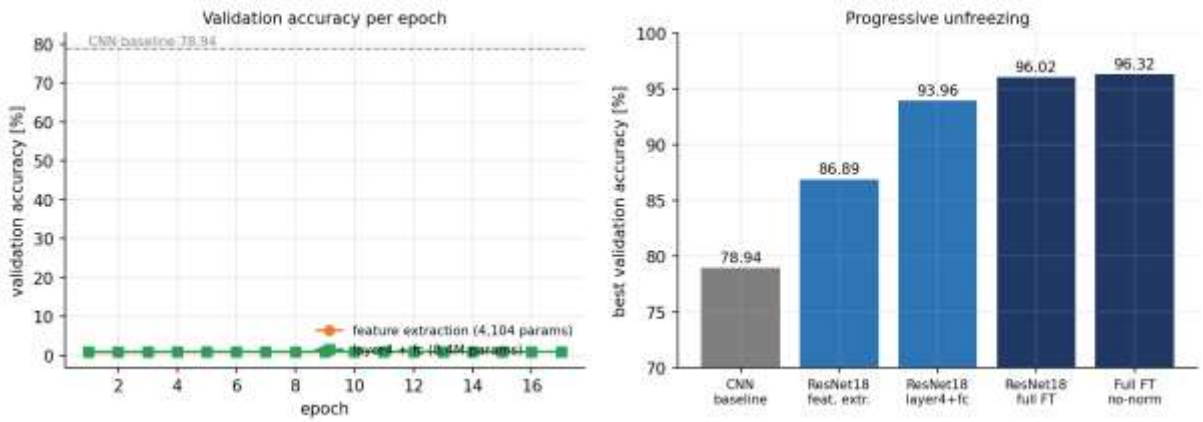
حذف نرمال‌سازی دقت را به ۹۶.۳۲٪ (اپوک ۸) رساند؛ اختلاف ۰.۳۰ واحد درصد، یعنی ۲ ≈ ۶۷۹ × ۰.۰۰۳۰ تصویر. این تفاوت را نمی‌توان شاهد قوی دانست. علت هم قابل فهم است: وقتی تمام لایه‌ها تنظیم می‌شوند و پس از `conv1` یک Batch Normalization وجود دارد، شبکه می‌تواند جابه‌جایی/مقیاس کانال‌های ورودی را جذب کند. با

این حال چون از وزن‌های پیش‌آموزش‌دیده ی ImageNet استفاده می‌شود، نرمال‌سازی ImageNet همچنان انتخاب استاندارد و سازگار است.

ث) مقایسه با CNN خط مبنا

جدول ۶۳ — مقایسه ی نهایی ResNet۱۸ با CNN خط مبنا

مدل	دقت اعتبارسنجی	Macro F۱
CNN Baseline	۷۸.۹۴٪	۷۷.۵۶٪
ResNet۱۸ Feature Extraction	۸۶.۸۹٪	۸۵.۹۱٪
ResNet۱۸ Layer۴ Fine-Tuning	۹۳.۹۶٪	۹۳.۷۷٪
ResNet۱۸ Full Fine-Tuning	**۹۶.۰۲٪**	**۹۵.۷۳٪**
ResNet۱۸ Full Fine-Tuning without normalization	۹۶.۳۲٪	—



شکل ۷ — چپ: منحنی دقت اعتبارسنجی Feature Extraction و Partial Fine-Tuning. راست: پیشرفت از CNN خط مبنا تا Full Fine-Tuning.

نسبت به CNN خط مبنا: Feature Extraction +۷.۹۵، Full Fine-Tuning حدود +۱۷.۰۸ واحد درصد بهتر است. بهبود صرفاً ناشی از جایگزینی لایه ی طبقه‌بند نیست؛ با باز کردن تدریجی بخش بیشتری از شبکه، بهبود افزایش می‌یابد.

نتیجه کلیدی: **نتیجه ی آزمایش ۱۰:** یادگیری انتقالی جهش اصلی پروژه است. بهترین راهبرد استاندارد Full Fine-Tuning با ۹۶.۰۲٪ (Macro F۱ = ۹۵.۷۳٪) است. چون دقت آموزش تقریباً ۱۰۰٪ می‌شود، Early Stopping و انتخاب بهترین اپوک ضروری است.

۴.۳ آزمایش ۱۱: حساسیت به seed و نوع افزایش داده

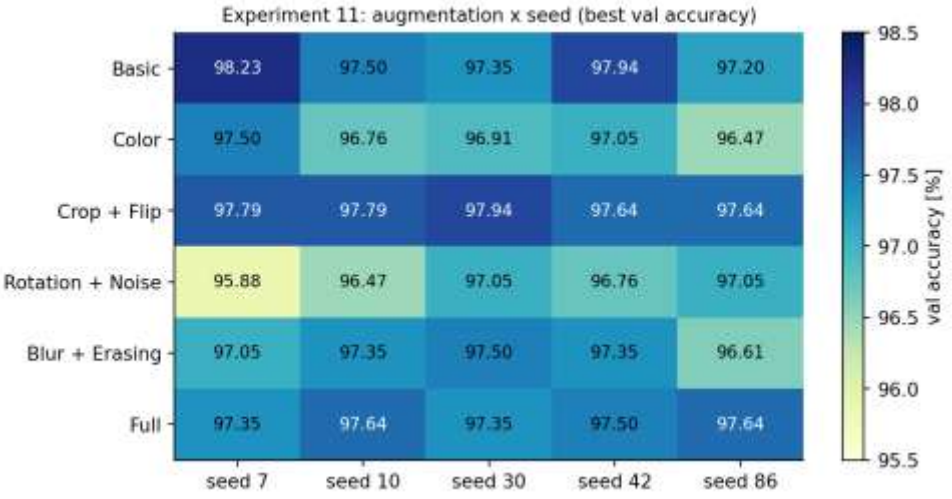
هدف: آموزش شبکه چند منبع تصادفی دارد (مقداردهی اولیه، ترتیب دسته‌ها، افزایش داده)، پس یک اجرا ممکن است نماینده ی خوبی نباشد. شش راهبرد افزایش داده با **پنج** (۸۶، ۴۲، ۳۰، ۱۰، ۷) **seed** ارزیابی شد (۳۰ اجرا) با ResNet۱۸ در حالت Full Fine-Tuning.

جدول ۶۴ — راهبردهای افزایش داده در آزمایش ۱۱ (پارامترهای دقیق در گزارش منبع ذکر نشده است)

راهبرد	عملیات	تعریف ریاضی/مفهومی
Basic	HFlip + Rotation	$x' = T(x)$ با چرخش $\phi \sim U(-a, a)$ و وارونگی افقی
Color	ColorJitter	تغییر تصادفی روشنایی/کنتراست/اشباع
Crop + Flip	RandomResizedCrop + HFlip	برش تصادفی با مساحت و نسبت ابعاد تصادفی و تغییر اندازه به ۲۲۴
Rotation + Noise	Rotation + Gaussian Noise	$x' = x + \varepsilon,\ \varepsilon \sim \mathcal{N}(0, \sigma^2)$
Blur + Erasing	Gaussian Blur + Random Erasing	کانولوشن با هسته ی گاوسی؛ جایگزینی یک مستطیل تصادفی با مقدار ثابت/نویز
Full	ترکیب چند روش	ترکیب ترتیبی چند تبدیل بالا

جدول ۶۵ — بهترین دقت اعتبارسنجی به‌ازای هر seed و راهبرد (٪)

افزایش داده	Seed ۷	Seed ۱۰	Seed ۳۰	Seed ۴۲	Seed ۸۶
Basic	۹۸.۲۳٪	۹۷.۵۰٪	۹۷.۳۵٪	۹۷.۹۴٪	۹۷.۲۰٪
Color	۹۷.۵۰٪	۹۶.۷۶٪	۹۶.۹۱٪	۹۷.۰۵٪	۹۶.۴۷٪
Crop + Flip	۹۷.۷۹٪	۹۷.۹۴٪	۹۷.۹۴٪	۹۷.۶۴٪	۹۷.۶۴٪
Rotation + Noise	۹۵.۸۸٪	۹۶.۴۷٪	۹۷.۰۵٪	۹۶.۷۶٪	۹۷.۰۵٪
Blur + Erasing	۹۷.۰۵٪	۹۷.۳۵٪	۹۷.۵۰٪	۹۷.۳۵٪	۹۶.۶۱٪
Full	۹۷.۳۵٪	۹۷.۶۴٪	۹۷.۳۵٪	۹۷.۵۰٪	۹۷.۶۴٪



شکل ۸ — نقشه ی حرارتی دقت اعتبارسنجی؛ ستون‌ها: seed، سطرها: راهبرد افزایش داده.

جدول ۶۶ — آمار تجمیعی روی پنج seed (محاسبه ی تکمیلی بر اساس جدول گزارش؛ std نمونه‌ای با $n-1$)

راهبرد	میانگین	انحراف معیار	کمینه	بیشینه	آزمون t زوجی در برابر Crop+Flip (p)
Crop + Flip	**۹۷.۷۶**	**۰.۱۳**	**۹۷.۶۴**	**۹۷.۹۴**	**—**
Basic	۹۷.۶۴	۰.۴۳	۹۷.۲۰	۹۸.۲۳	۰.۶۰۲
Full	۹۷.۵۰	۰.۱۵	۹۷.۳۵	۹۷.۶۴	۰.۰۷۲
Blur + Erasing	۹۷.۱۷	۰.۳۵	۹۶.۶۱	۹۷.۵۰	۰.۰۱۱
Color	۹۶.۹۴	۰.۳۸	۹۶.۴۷	۹۷.۵۰	۰.۰۰۸
Rotation + Noise	۹۶.۶۴	۰.۴۹	۹۵.۸۸	۹۷.۰۵	۰.۰۰۸

جدول ۶۷ — جدول میانگین به‌همان شکلی که در گزارش منبع آمده است

افزایش داده	میانگین دقت اعتبارسنجی	بهترین نتیجه	بدترین نتیجه
Basic	۹۷.۶۴٪	۹۸.۲۳٪	۹۷.۲۰٪
Crop + Flip	۹۷.۷۶٪	۹۷.۹۴٪	۹۷.۶۴٪
Full	۹۷.۵۰٪	۹۷.۶۴٪	۹۷.۳۵٪
Blur + Erasing	۹۷.۱۷٪	۹۷.۵۰٪	۹۶.۶۱٪
Color	۹۶.۹۴٪	۹۷.۵۰٪	۹۶.۴۷٪
Rotation + Noise	۹۶.۶۴٪	۹۷.۰۵٪	۹۵.۸۸٪

تحلیل. Crop + Flip بالاترین میانگین (۹۷.۷۶٪) و کوچک‌ترین پراکندگی (از ۹۷.۶۴٪ تا ۹۷.۹۴٪، دامنه ۰.۳۰ واحد؛ انحراف معیار ۰.۱۳) را دارد. Basic بالاترین نتیجه ی منفرد (۹۸.۲۳٪ با ۷ seed) را دارد ولی ناپایدارتر است (۹۷.۲۰٪ تا ۹۸.۲۳٪). راهبرد Full با وجود ترکیب چند تبدیل (میانگین ۹۷.۵۰٪) از ساده‌ترها بهتر نیست؛ بنابراین «بیشتر بودن تبدیل‌ها» لزوماً بهتر نیست (تغییراتی که با توزیع واقعی تصاویر خودرو نمی‌خوانند، فقط نوفه می‌افزایند). Rotation + Noise ضعیف‌ترین است (میانگین ۹۶.۶۴٪، کمینه ۹۵.۸۸٪).

اثر seed. اثر seed (انحراف معیار ۰.۱۳ تا ۰.۴۹) از اثر نوع افزایش داده بین گروه برتر و پایین (حدود ۱.۱ واحد) کوچک‌تر ولی در گروه‌های برتر هم‌اندازه ی اختلاف میانگین‌هاست. پس ارزیابی چند-seed برای گزارش صادقانه ضروری است. همچنین از این آزمایش به بعد seed = ۷ انتخاب شد؛ چون seed ۷ بهترین نتیجه ی منفرد (Basic: ۹۸.۲۳٪) را داد، دقت‌های گزارش‌شده با seed ۷ قدری **خوش‌بینانه** هستند (سوگیری انتخاب). نتیجه ی آزمون (Test) در بخش ۵ برای کاهش این نگرانی اهمیت دارد.

اجرای آزمایش ۱۱ دقت اعتبارسنجی را به حدود ۹۷–۹۸٪ رساند، یعنی ≈ ۱.۵–۲ واحد بالاتر از Full Fine-Tuning بدون افزایش داده (۹۶.۰۲٪). هرچند تنظیمات دیگر نیز بین این دو آزمایش متفاوت است، افزایش داده یکی از عوامل مهم این بهبود است. در پایان متن منبع جمله ی آخر ناقص مانده است؛ مفهوم آن این است که انتخاب نهایی باید «پس از تثبیت همه ی انتخاب‌های مدل و افزایش داده، تنها یک بار روی مجموعه ی نگه‌داشته ی آزمون» تأیید شود.

نتیجه کلیدی: نتیجه ی آزمایش ۱۱: افزایش داده ی هندسیِ ملایم (Crop+Flip یا Basic) بهتر از ترکیب‌های پیچیده است؛ نوع افزایش داده مهم‌تر از seed است. Crop+Flip پایدارترین، و Basic بهترین نتیجه ی منفرد را دارد.

۵. فاز سوم: بررسی Letterbox و Dropout در مدل نهایی (آزمایش‌های ۱۲ و ۱۳)

پس از رسیدن به ResNet۱۸ با دقت ≈ ۹۸٪، پرسش اصلی «تأثیر پیش‌پردازش Letterbox در آموزش» و «لزوم Dropout» بود. در هر دو آزمایش، دو پیکربندی فقط در **نحوه ی تغییر اندازه** تفاوت دارند: Letterbox (حفظ نسبت ابعاد با حاشیه ی سیاه) در برابر تغییر اندازه ی مستقیم ۲۲۴ × ۲۲۴ (Non-Letterbox). مبنای ریاضی هر دو در بخش ۲.۲ توضیح داده شد.

۵.۱ تنظیمات آزمایش‌های ۱۲ و ۱۳

جدول ۶۸ — پیکربندی آزمایش ۱۲ (با ۰.۳ = Dropout)

پارامتر	مقدار
Architecture	ResNet۱۸
Pretrained weights	ImageNet
Training strategy	Full fine-tuning
Number of classes	۸
Input size	۲۲۴ × ۲۲۴
Trainable layers	All layers
Dropout	۰.۳
Optimizer	Adam
Backbone learning rate	۱e-۴
Head learning rate	۱e-۴
Weight decay	۱e-۴
Scheduler	ReduceLROnPlateau
Scheduler mode	max
Scheduler factor	۰.۰۳
Scheduler patience	۱۰
Batch size	۳۲
Maximum epochs	۵۰۰
Random seed	۷
Normalization	ImageNet

جدول ۶۹ — پیکربندی آزمایش ۱۳ (بدون Dropout)

پارامتر	مقدار
Architecture	ResNet۱۸
Pretrained weights	ImageNet
Training strategy	Full fine-tuning
Number of classes	۸
Input size	۲۲۴ × ۲۲۴
Trainable layers	All layers
Dropout	None
Optimizer	Adam
Backbone learning rate	۱e-۴
Head learning rate	۱e-۴
Weight decay	۱e-۴
Scheduler	ReduceLROnPlateau
Scheduler factor	۰.۰۳
Seed	۷
Batch size	۳۲
Maximum epochs	۱۵۰
Normalization	ImageNet mean and standard deviation

افزایش داده در هر دو: `RandomHorizontalFlip(0.5)` و `RandomRotation(±10°)` پس از (یا بدون) Letterbox، سپس `ToTensor` و نرمال‌سازی ImageNet. مدل از وزن‌های ImageNet مقداردهی می‌شود و لایه ی آخر شامل (در آزمایش ۱۲) `Dropout(0.3)` و سپس `Linear(512→8)` است. در زمان ارزیابی Dropout به‌صورت خودکار خاموش می‌شود. بهترین مدل بر اساس **دقت اعتبارسنجی** (نه آخرین اپوک) انتخاب می‌شود. زمان‌بند با `mode=max` بر دقت اعتبارسنجی نظارت می‌کند: اگر در ۱۰ (یا ۷۵) اپوک بهبود نیابد نرخ یادگیری در ضریب ۰.۰۳ ضرب می‌شود. آموزش روی GPU (CUDA) انجام شد.

۵.۲ آزمایش ۱۲: Letterbox در برابر Non—Letterbox با Dropout

جدول ۷۰ — نتایج آزمایش ۱۲

مدل	پیش‌پردازش	بهترین دقت اعتبارسنجی
ResNet۱۸ Full Fine-tuning	Letterbox	۹۷.۷۹٪

مدل	پیش‌پردازش	بهترین دقت اعتبارسنجی
ResNet۱۸ Full Fine-tuning	**Non-Letterbox**	**۹۸.۳۸٪**

مدل **Non-Letterbox** به ۹۸.۳۸٪ (اپوک ۷۰) و مدل **Letterbox** به ۹۷.۷۹٪ (اپوک ۳۷) رسید؛ اختلاف ۹۸.۳۸ − ۹۷.۷۹ = ۰.۵۹ واحد درصد. Early Stopping برای Letterbox در اپوک ۱۱۲ و برای Non-Letterbox در اپوک ۱۴۵ فعال شد. دقت آموزش هر دو به ≈ ۱۰۰٪ می‌رسد، ولی دقت اعتبارسنجی پس از رسیدن به بهترین مقدار فقط نوسان می‌کند؛ یعنی باید بهترین Checkpoint (نه آخرین اپوک) استفاده شود.

جدول ۷۱ — عملکرد به‌تفکیک کلاس برای مدل Non-Letterbox آزمایش ۱۲

کلاس	Precision	Recall	F۱
ambulance	۱.۰۰	۱.۰۰	۱.۰۰
autobus	۱.۰۰	۱.۰۰	۱.۰۰
kamyun	۰.۹۹	۰.۹۸	۰.۹۸
kamyunet	۰.۹۶	۰.۹۹	۰.۹۷
minibus	۰.۹۸	۰.۹۸	۰.۹۸
savari	۰.۹۷	۱.۰۰	۰.۹۹
taxi	۰.۹۹	۰.۹۸	۰.۹۸
vanet	۱.۰۰	۰.۹۰	۰.۹۵

معیارهای کلی: Macro Recall = ۹۷.۸۴٪، Macro Precision = ۹۸.۵۵٪، Accuracy = ۹۸.۳۸٪، Macro F۱ = ۹۸.۱۵٪، Weighted F۱ = ۹۸.۳۷٪. ضعیف‌ترین Recall متعلق به vanet (حدود ۰.۹۰) است ولی Precision آن ۱.۰۰ است؛ پس هر پیش‌بینی vanet قابل اعتماد است ولی ≈ ۱۰٪ نمونه‌های واقعی vanet به کلاس‌های دیگر می‌روند.

جدول ۷۲ — مقایسه ی مستقیم Letterbox و Non-Letterbox (آزمایش ۱۲)

پیش‌پردازش	بهترین دقت اعتبارسنجی
Letterbox	۹۷.۷۹٪
Non-Letterbox	**۹۸.۳۸٪**

اثر Dropout و تحلیل بیش‌برازش

با وجود $p = ۰.۳$ مدل همچنان به دقت بسیار بالا می‌رسد؛ یعنی Dropout مانع یادگیری ویژگی‌های تفکیک‌پذیر نشد. دقت آموزش به ≈ ۱۰۰٪ می‌رسد، پس ظرفیت مدل بسیار بالاست و مجموعه ی منظم‌سازها (Dropout،

weight decay، افزایش داده، زمان‌بند، Early Stopping) رفتار را کنترل می‌کنند. مقایسه با آزمایش بدون Dropout باید محتاطانه انجام شود؛ زیرا پیکربندی‌ها در جزئیات دیگر هم ممکن است فرق کنند (مثلاً patience زمان‌بند و حداکثر اپوک).

ابزار Grad–CAM برای تحلیل خطا

برای تحلیل کیفی نمونه‌های اشتباه از **Grad-CAM** استفاده شد. این روش با گرادیان امتیاز کلاس c نسبت به نقشه‌های ویژگی آخرین لایه ی کانولوشنی (A^k در `layer4`) وزن هر نقشه را به‌دست می‌آورد:

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c\, A^k\right)$$

نقشه ی L^c به اندازه ی تصویر بزرگ و روی آن رنگ‌آمیزی می‌شود تا نشان دهد کدام نواحی (بدنه، چرخ، پس‌زمینه یا زمینه) بیشترین سهم را در تصمیم داشته‌اند. این تحلیل مکمل معیارهای عددی است. خروجی تصویری Grad-CAM در گزارش منبع گنجانده نشده و بنابراین ادعای خاصی درباره ی نواحی مورد توجه مدل نمی‌توان کرد.

۵.۳ آزمایش ۱۳: Letterbox در برابر Non–Letterbox بدون Dropout

جدول ۷۳ — نتایج آزمایش ۱۳

مدل	پیش‌پردازش	بهترین دقت اعتبارسنجی
ResNet۱۸ Full Fine-tuning	Letterbox	۹۸.۰۹٪
ResNet۱۸ Full Fine-tuning	**Non-Letterbox**	**۹۸.۳۸٪**

Non-Letterbox: **۹۸.۳۸٪** (اپوک ۱۲۲)؛ Letterbox: **۹۸.۰۹٪** (اپوک ۲۰). اختلاف ۰.۲۹ واحد درصد، یعنی حدود ۲ تصویر از ۶۷۹. مدل Non-Letterbox آموزش طولانی‌تری (۱۲۲ اپوک تا بهترین نقطه) نیاز داشت.

جدول ۷۴ — مقایسه ی مستقیم آزمایش ۱۳

معیار	Letterbox	Non-Letterbox
Best validation accuracy	۹۸.۰۹٪	۹۸.۳۸٪
Difference	—	+۰.۲۹ percentage points

جدول ۷۵ — عملکرد به‌تفکیک کلاس برای مدل Non-Letterbox آزمایش ۱۳ (بدون Dropout)

کلاس	Precision	Recall	F۱

کلاس	Precision	Recall	F۱
ambulance	۱.۰۰	۰.۹۹	۰.۹۹
autobus	۰.۹۹	۰.۹۹	۰.۹۹
kamyun	۰.۹۸	۰.۹۹	۰.۹۹
kamyunet	۰.۹۸	۰.۹۸	۰.۹۸
minibus	۰.۹۹	۰.۹۸	۰.۹۸
savari	۰.۹۸	۱.۰۰	۰.۹۹
taxi	۰.۹۸	۰.۹۹	۰.۹۸
vanet	۰.۹۷	۰.۹۳	۰.۹۵

معیارهای کلی: Macro Recall = ۹۷.۹۵٪، Macro Precision = ۹۸.۳۷٪، Accuracy = ۹۸.۳۸٪، Weighted F۱ = ۹۸.۳۷٪، F۱ = ۹۸.۱۵٪. باز هم vanet با Recall = ۰.۹۳ ضعیف‌ترین است، ولی F۱ برابر ۰.۹۵ آن از CNN پایه (۰.۵۸) بسیار بهتر است. حذف Dropout مانع تعمیم قوی نشد؛ پس ResNet۱۸ پیش‌آموزش‌دیده با افزایش داده، weight decay و زمان‌بند به‌تنهایی عالی عمل می‌کند.

۵.۴ Dropout یا بدون Dropout؟ نقش مجموعه ی آزمون

جدول ۷۶ — مقایسه ی Dropout در برابر بدون Dropout (با دقت آزمون)

مدل	پیش‌پردازش	Dropout	بهترین دقت اعتبارسنجی	دقت آزمون
ResNet۱۸ Full Fine-tuning	Letterbox	۰.۳	۹۷.۷۹٪	—
ResNet۱۸ Full Fine-tuning	Letterbox	۰.۰	۹۸.۰۹٪	—
ResNet۱۸ Full Fine-tuning	Non-Letterbox	۰.۳	۹۸.۳۸٪	۹۷.۵٪
ResNet۱۸ Full Fine-tuning	**Non-Letterbox**	**۰.۰**	**۹۸.۳۸٪**	**۹۸.۵٪**

برای مدل‌های Non-Letterbox دقت اعتبارسنجی هر دو **۹۸.۳۸٪** است (بنابراین اعتبارسنجی نمی‌تواند تمایز قائل شود) ولی دقت آزمون با Dropout برابر ۹۷.۵٪ و بدون Dropout برابر ۹۸.۵٪ است؛ یعنی ۱.۰ واحد درصد بیشتر برای بدون Dropout. آزمون ۴۰۰ تصویر دارد؛ پس ۹۸.۵٪ یعنی ۳۹۴ درست (۶ خطا) و ۹۷.۵٪ یعنی ۳۹۰ درست (۱۰ خطا): اختلاف ۴ تصویر.

۵.۵ معناداری آماری: تفاوت‌ها چقدر بزرگ‌اند؟

خطای استاندارد یک نسبت $\hat{p}$ روی n نمونه برابر $SE = \sqrt{\hat{p}(۱ - \hat{p})/n}$ است و بازه ی اطمینان ۹۵٪ تقریباً $\hat{p} \pm ۱.۹۶\,SE$ (در جدول زیر با روش Wilson). برای اعتبارسنجی ($n = ۶۷۹$) و آزمون ($n = ۴۰۰$):

جدول ۷۷ — بازه ی اطمینان دقت‌ها

مورد	n	درست	دقت	بازه ی اطمینان ۹۵٪ (Wilson)
اعتبارسنجی – Non-Letterbox (۹۸.۳۸٪)	۶۷۹	۶۶۸	۹۸.۳۸٪	۹۷.۱٪ تا ۹۹.۱٪
اعتبارسنجی – Letterbox (۹۷.۷۹٪)	۶۷۹	۶۶۴	۹۷.۷۹٪	۹۶.۴٪ تا ۹۸.۷٪
آزمون – بدون Dropout (۹۸.۵٪)	۴۰۰	۳۹۴	۹۸.۵٪	۹۶.۸٪ تا ۹۹.۳٪
آزمون – با Dropout (۹۷.۵٪)	۴۰۰	۳۹۰	۹۷.۵٪	۹۵.۵٪ تا ۹۸.۷٪

تحلیل انتقادی: بازه‌ها به‌شدت هم‌پوشان‌اند. تفاوت‌های ۰.۲۹، ۰.۵۹ و حتی ۱.۰ واحد درصد (۲ تا ۴ تصویر) **در محدوده ی نوسان نمونه‌گیری** قرار دارند. به همین دلیل نتیجه ی علمی درست این است: «Letterbox مزیت **قابل‌اثباتی** نشان نداد» نه اینکه «Letterbox بدتر است»؛ و برتری بدون Dropout نیز مشاهده‌ای است که برای تأیید قطعی به seedهای بیشتر و آزمون بزرگ‌تر نیاز دارد. برای بررسی دقیق‌تر باید پیش‌بینی‌های جفت‌شده (مثلاً آزمون McNemar) روی همان تصاویر بررسی شود.

۵.۶ چرا Letterbox سودی نداشت؟ (جمع‌بندی تحلیلی)

- **ResNet از قبل به کشیدگی مقاوم است:** آموزش ImageNet با نسبت ابعاد تصادفی [۳/۴,۴/۳] انجام شده و کشیدگی متوسط ۱.۲۳ داده ی ما در همین محدوده است (بخش ۲.۲).
- **هزینه ی حاشیه:** در Letterbox حدود ۱۹٪ (و تا ۶۲٪ در تصاویر بسیار باریک) از قاب بدون اطلاعات است؛ رزولوشن مؤثر خودرو کم می‌شود.
- **الگوی ثابت حاشیه و برش‌ها:** حاشیه ی سیاه پس از نرمال‌سازی مقدار منفی ثابت (−۲.۱، −۲.۰، −۱.۸) دارد و چرخش حاشیه‌ها را هم می‌چرخاند؛ این الگوها هیچ اطلاعات کلاسی ندارند.
- **اطلاعات خودرو کافی است:** مشخصه‌های تفکیک‌کننده (شکل بدنه، پنجره‌ها، چراغ‌ها) در هر دو حالت حفظ می‌شوند.
- **محدودیت:** این‌ها تفسیرند و در آزمایش‌ها مستقیماً اثبات نشده‌اند؛ تنها نتیجه ی مستقیم این است که Letterbox دقت را بهبود نداد.

۵.۷ انتخاب مدل نهایی

بر اساس نتایج اعتبارسنجی و آزمون، **ResNet۱۸ با Full Fine-Tuning، بدون Letterbox و بدون Dropout** انتخاب شد: دقت اعتبارسنجی ۹۸.۳۸٪ و دقت آزمون ۹۸.۵٪. Checkpoint مربوطه `resnet18_full_finetuning_non_letterbox_best.pt` است.

۶. فاز چهارم: تشخیص نمونه‌های خارج از توزیع (آزمایش ۱۴)

۶.۱ مسئله و انگیزه

طبقه‌بند ResNet۱۸ فقط ۸ کلاس را می‌شناسد. اگر تصویر وسیله ی ناشناخته‌ای مانند **Neysan** (**نیسان**) به آن داده شود، ناچار است یکی از ۸ کلاس را انتخاب کند و ممکن است با اطمینان بالا اشتباه کند. بنابراین باید بتوان ورودی‌هایی را که با توزیع آموزش فاصله دارند (Out-of-Distribution یا OOD) شناسایی کرد، بدون اینکه Neysan به‌عنوان کلاس نهم به مدل اضافه شود. در این آزمایش دو روش بررسی شد:

۶.۲ مبانی ریاضی

Energy Score از لوجیت‌های مدل محاسبه می‌شود ($T = ۱$):

$$E(x) = -T \log \sum_{i=۱}^{۸} \exp\left(\frac{f_i(x)}{T}\right)$$

انرژی بالاتر یعنی ورودی با توزیع شناخته‌شده کمتر سازگار است (OOD بودن محتمل‌تر). **فاصله ی ماهالانوبیس** روی بردار ویژگی ۵۱۲ بعدی پیش از لایه ی طبقه‌بند محاسبه می‌شود؛ برای هر کلاس k میانگین μ_k و یک کوواریانس مشترک Σ (با **Ledoit–Wolf shrinkage** برای پایداری معکوس‌پذیری) تخمین زده می‌شود:

$$D_k(x) = \sqrt{(x-\mu_k)^\top \Sigma^{-۱} (x-\mu_k)}, \qquad D(x) = \min_k D_k(x)$$

و آستانه‌ها فقط با **صدک ۹۵ام** امتیازهای اعتبارسنجی داده ی شناخته‌شده تعیین می‌شوند (نه با Neysan). جزئیات کامل این مبانی (از جمله اشتقاق ماهالانوبیس و ارتباط انرژی با softmax) در **گزارش دوم** (**فایل نهایی**) آمده است. قاعده ی تصمیم در دو لایه: اگر هر دو آشکارساز OOD بگویند ⇒ `unknown`؛ اگر فقط یکی ⇒ `needs_review`؛ اگر هیچ‌کدام نگویند ولی اطمینان Softmax کمتر از ۷۰٪ باشد ⇒ `needs_review`؛ در غیر این صورت برای نمونه ی Neysan واقعی `known_failure` (آشکارساز شکست خورده).

۶.۳ تنظیمات و داده

جدول ۷۸ — تنظیمات آزمایش ۱۴

مورد	مقدار

مورد	مقدار
Checkpoint	resnet۱۸_full_finetuning_non_letterbox_best.pt
داده‌های شناخته‌شده	۲,۷۱۳ آموزش + ۶۷۹ اعتبارسنجی
داده‌های ناشناخته (Neysan)	۵۸۶ تصویر (جدا از آموزش و اعتبارسنجی)
ابعاد ویژگی	۵۱۲
پیش‌پردازش (طبق گزارش)	Letterbox ۲۲۴×۲۲۴ + نرمال‌سازی ImageNet
آستانه ی Energy	−۵.۹۰۵۷
آستانه ی Mahalanobis	۲۰.۱۵۱۲
آستانه ی اطمینان	۷۰٪

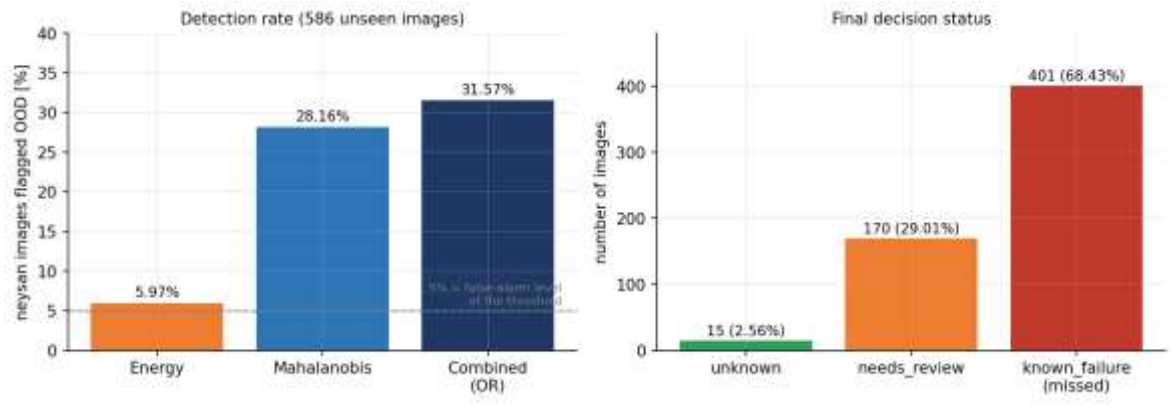
۶.۴ نتایج

جدول ۷۹ — نرخ شناسایی Neysan توسط هر آشکارساز

روش آشکارسازی	Neysan تشخیص داده‌شده به‌عنوان OOD	تعداد
Energy	۵.۹۷٪	۳۵ از ۵۸۶
Mahalanobis	۲۸.۱۶٪	۱۶۵ از ۵۸۶
ترکیبی (Energy یا Mahalanobis)	**۳۱.۵۷٪**	**۱۸۵ از ۵۸۶**

جدول ۸۰ — وضعیت نهایی ۵۸۶ تصویر Neysan

وضعیت	تعداد تصاویر	درصد
unknown	۱۵	۲.۵۶٪
needs_review	۱۷۰	۲۹.۰۱٪
OOD missed / known_failure	۴۰۱	۶۸.۴۳٪
جمع	**۵۸۶**	**۱۰۰٪**



شکل ۹ — نتایج آزمایش ۱۴: نرخ شناسایی هر روش (چپ) و توزیع وضعیت نهایی (راست).

بررسی سازگاری درونی اعداد

اگر A مجموعه ی شناسایی‌شدگان توسط Energy و B توسط Mahalanobis باشد: $|A \cup B| = |A| + |B| - |A \cap B| \Rightarrow ۱۸۵ = ۳۵ + ۱۶۵ - |A \cap B| = ۱۵$. این عدد دقیقاً برابر تعداد `unknown` (هر دو آشکارساز) است؛ و $۱۵ + ۱۷۰ = ۱۸۵$ برابر مجموع `unknown` و `needs_review` است. یعنی اعداد گزارش از نظر منطقی سازگارند.

تفسیر

- **Energy تقریباً بی‌اثر است:** نرخ شناسایی ۵.۹۷٪ تقریباً برابر نرخ هشدار کاذبی است که آستانه ی صدک ۹۵ روی داده ی شناخته‌شده می‌دهد (≈ ۵٪)؛ یعنی Energy نمونه‌های Neysan را از نمونه‌های شناخته‌شده تفکیک نمی‌کند.
- **Mahalanobis معنادار است:** ۲۸.۱۶٪ به‌وضوح بالاتر از سطح ۵٪ است؛ فضای ویژگی اطلاعات بیشتری از لوجیت‌ها دارد.
- **ترکیب کمک محدودی می‌کند:** ۳۱.۵۷٪ فقط ۳.۴ واحد از Mahalanobis تنهاست.
- **نیمی از نتیجه «بازبینی» است:** ۲۹.۰۱٪ در `needs_review` و تنها ۲.۵۶٪ در `unknown` قرار می‌گیرد؛ سامانه یک صافی غربالگری است نه تصمیم‌گیر قطعی.

تحلیل خطا: همه ی نمونه‌های جاافتاده vanet پیش‌بینی شده‌اند

جدول ۸۱ — تحلیل ۴۰۱ تصویر Neysan که آشکارساز آن‌ها را از دست داد

کلاس پیش‌بینی‌شده	تعداد	میانگین اطمینان	کمینه ی اطمینان	بیشینه ی اطمینان
vanet	۴۰۱	۹۹.۹۵٪	۹۷.۰۸٪	۱۰۰.۰۰٪

هر ۴۰۱ تصویر جاافتاده به‌عنوان **vanet** و با اطمینان بسیار بالا پیش‌بینی شده‌اند. (vanet = «وانت» و Neysan = «نیسان»، یک وانت/خودروی باری سبک؛ شباهت ظاهری طبیعی است.) یعنی مدل درباره ی این تصاویر «نامطمئن» نیست؛ آن‌ها را با اطمینان در ناحیه ی vanet فضای ویژگی جا می‌دهد. بنابراین **آستانه‌گذاری روی اطمینان Softmax به‌تنهایی کارساز نیست** (کمینه ی اطمینان ۹۷.۰۸٪ است و قاعده ی «اطمینان < ۷۰٪» هرگز برای این نمونه‌ها فعال نمی‌شود).

توضیح محتمل: شبکه ی آموزش‌دیده با تابع هزینه ی نظارتی ویژگی‌هایی می‌سازد که فقط برای تفکیک ۸ کلاس شناخته‌شده بهینه‌اند؛ هر ورودی نزدیک به خوشه ی vanet (حتی اگر کلاس تازه‌ای باشد) به همان خوشه نگاشت می‌شود. همچنین vanet کم‌نمونه‌ترین کلاس است (۲۰۵ تصویر) و مرز آن با «ناشناخته» تنگ است.

۷. جمع‌بندی و نتیجه‌گیری کلی

۷.۱ جدول کامل آزمایش‌ها (Result.json)

تمام اجراهای ثبت‌شده در `experiment_results.json` در جدول زیر آمده‌اند. ستون «#» شماره ی ردیف در این فایل است و با شماره ی آزمایش‌های ۰۱ تا ۱۴ فرق دارد (نگاشت در جدول بعد).

جدول ۸۳ — جدول جامع اجراهای ثبت‌شده در Result.json (ستون خطای ردیف ۱۲ قابل‌مقایسه با CE نیست)

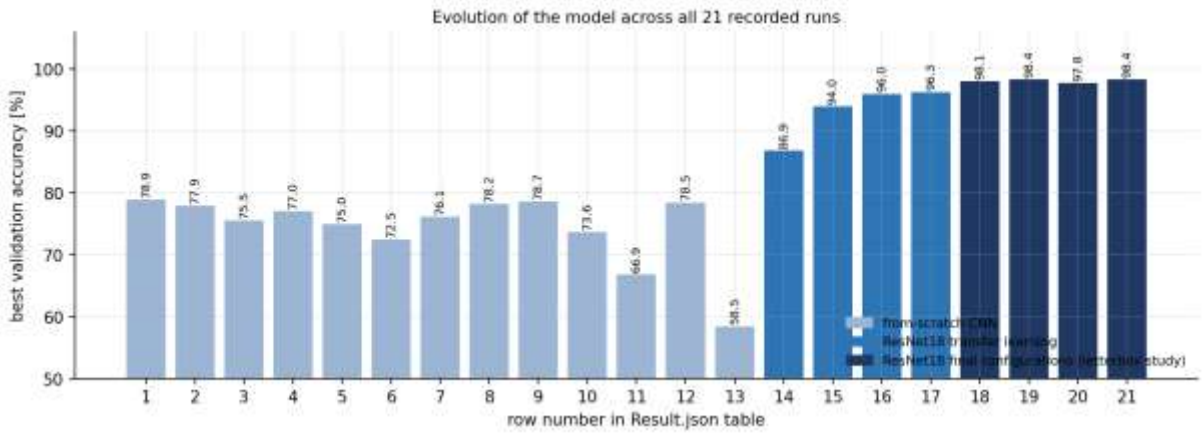
#	کلید آزمایش	بهترین اپوک	دقت	خطا	Seed	پیکربندی کلیدی
۱	baseline_cnn	۹	۷۸.۹۴٪	۱.۱۱۶۴	۴۲	SimpleCNN, no dropout, no GAP
۲	augmentation	۱۰	۷۷.۹۱٪	۰.۷۲۴۴	۴۲	HFlip + Rotation(۱۰) + ColorJitter(۰.۲)
۳	dropout_۰۲	۹	۷۵.۵۵٪	۱.۰۷۵۰	۴۲	dropout = ۰.۲
۴	dropout_۰۳	۹	۷۷.۰۳٪	۰.۹۳۲۶	۴۲	dropout = ۰.۳
۵	dropout_۰۵	۱۰	۷۴.۹۶٪	۰.۸۸۰۹	۴۲	dropout = ۰.۵
۶	avg_pooling	۵	۷۲.۴۵٪	۰.۹۵۳۳	۴۲	MaxPool۲d → AvgPool۲d
۷	weight_decay_۰	۱۰	۷۶.۱۴٪	۱.۳۴۵۲	۴۲	AdamW, wd = ۰
۸	weight_decay_۱e۴	۱۰	۷۸.۲۰٪	۱.۱۴۱۸	۴۲	AdamW, wd = ۱e-۴
۹	scheduler_reduce_on_plateau	۸	۷۸.۶۵٪	۱.۰۲۰۷	۴۲	ReduceLROnPlateau (factor ۰.۵, patience ۲)
۱۰	balanced_batches_standard_shuffle	۳	۷۳.۶۴٪	۰.۹۰۰۵	۴۲	Simulated imbalance (۳۰٪ of ambulance/autobus/kamyun), shuffle=True
۱۱	balanced_batches_sampler	۷	۶۶.۸۶٪	۱.۶۹۱۳	۴۲	Same imbalance, BalancedBatchSampler
۱۲	bce_loss	۵	۷۸.۴۹٪	❋۰.۱۷۷۶	۴۲	BCEWithLogitsLoss
۱۳	gap	۲۲	۵۸.۴۷٪	۱.۱۷۹۱	۴۲	Global Average Pooling, up to ۵۰ epochs
۱۴	resnet۱۸_feature_extraction	۱۱	۸۶.۸۹٪	۰.۴۲۰۳	۴۲	ResNet۱۸, frozen backbone, fc only
۱۵	resnet۱۸_finetuning	۱۲	۹۳.۹۶٪	۰.۲۷۰۳	۴۲	layer۴ (lr ۱e-۴) + fc (lr ۱e-۳)
۱۶	resnet۱۸_full_finetuning	۳	۹۶.۰۲٪	۰.۱۳۲۶	۴۲	All layers, backbone lr ۱e-۴, head lr ۱e-۳
۱۷	resnet۱۸_full_finetuning_non_normalized	۸	۹۶.۳۲٪	۰.۱۳۷۹	۴۲	Same as #۱۶, no ImageNet normalization
۱۸	resnet۱۸_full_finetuning_letterbox_no_dropout	۲۰	۹۸.۰۹٪	۰.۰۸۶۳	۷	Letterbox + HFlip + Rotation(۱۰),head lr ۱e-۴, factor ۰.۰۳, patience ۷۵
۱۹	**resnet۱۸_full_finetuning_non_letterbox_no_dropout**	**۱۲۲**	**۹۸.۳۸٪**	**۰.۰۸۸۷**	**۷**	**Same as #۱۸, without letterbox**
۲۰	resnet۱۸_full_finetuning_letterbox_dropout	۳۷	۹۷.۷۹٪	۰.۱۰۹۹	۷	Letterbox + HFlip + Rotation(۱۰),dropout (p = ۰.۳), head lr ۱e-۴, factor ۰.۰۳, patience ۱۰
۲۱	**resnet۱۸_full_finetuning_non_letterbox_dropout**	**۷۰**	**۹۸.۳۸٪**	**۰.۰۷۸۶**	**۷**	**HFlip + Rotation(۱۰),dropout (p = ۰.۳), head lr ۱e-۴, factor ۰.۰۳, patience ۱۰**

(❋) خطای ردیف ۱۲ مربوط به BCE است و با خطای CrossEntropy ردیف‌های دیگر قابل مقایسه نیست.

جدول ۸۴ — نگاشت شماره ی آزمایش‌ها به ردیف‌های Result.json

آزمایش	ردیف‌های Result.json	توضیح
۰۱ – ۰۳	۱، ۲، ۳–۵	خط مبنا، افزایش داده، Dropout (سه مقدار)
۰۴ – ۰۶	۶، ۷–۸، ۹	Avg Pooling، weight decay (دو مقدار)، زمان‌بند
۰۷ – ۰۹	۱۰–۱۱، ۱۲، ۱۳	نمونه‌گیر متوازن (دو اجرا)، BCE (CE جداگانه ثبت نشده)، GAP
۱۰	۱۴ – ۱۷	ResNet۱۸: FE، layer۴، Full، Full بدون نرمال‌سازی
۱۱	—	۳۰ اجرا (۶ راهبرد × ۵ seed)؛ در Result.json نیامده‌اند
۱۲ – ۱۳	۱۸ – ۲۱	Letterbox و Non-Letterbox با و بدون Dropout
۱۴	—	آزمایش OOD؛ دقت ندارد، در جدول نیامده

۷.۲ تکامل مدل: داستان پروژه



شکل ۱۰ — تکامل بهترین دقت اعتبارسنجی در ۲۱ اجرای ثبت‌شده.

مسیر پروژه را می‌توان در چند گام روایت کرد:

- **گام ۰ – داده:** پاک‌سازی ۲۳ تصویر تکراری و حذف نشت؛ ۳,۳۹۲ تصویر؛ تقسیم Stratified.
- **گام ۱ – CNN پایه (۷۸.۹۴٪):** بیش‌برازش شدید (شکاف ۲۱.۱ واحد) و اشتباهات ساختاری kamyun/kamyunet و vanet/savari.
- **گام ۲ – تلاش برای بهبود (آزمایش‌های ۰۲–۰۹):** هیچ ترفندی از سقف ≈ ۸۰٪ عبور نکرد؛ GAP به ۵۸.۴۷٪ سقوط کرد؛ نمونه‌گیر متوازن بدتر شد؛ CE بهتر از BCE بود.
- **گام ۳ – جهش با یادگیری انتقالی (آزمایش ۱۰):** ۸۶.۸۹٪ ⟶ ۹۳.۹۶٪ ⟶ ۹۶.۰۲٪ با باز کردن تدریجی لایه‌ها.

- **گام ۴ – بهینه‌سازی افزایش داده و seed (آزمایش ۱۱):** ≈ ۹۷.۵–۹۸.۲٪ با افزایش داده هندسی ملایم.
- **گام ۵ – مدل نهایی (آزمایش‌های ۱۲–۱۳):** بدون Letterbox و بدون Dropout: ۹۸.۳۸٪ اعتبارسنجی و ۹۸.۵٪ آزمون.
- **گام ۶ – دنیای باز (آزمایش ۱۴):** تشخیص Neysan فقط ۳۱.۵۷٪؛ همه ی موارد جاافتاده vanet با اطمینان بالا.

۷.۳ درس‌های اصلی

- **بازنمایی بر ترفند غلبه می‌کند:** جهش ۱۷ واحد درصدی از ResNet۱۸ پیش‌آموزش‌دیده بسیار بزرگ‌تر از هر ترفند منظم‌سازی روی CNN ساده بود.
- **ظرفیت و منظم‌سازی باید متناسب باشند:** GAP با حذف شدید ظرفیت (در یک شبکه ی کم‌عمق) کم‌برازش ایجاد کرد؛ کاهش پارامتر لزوماً تعمیم نمی‌آورد.
- **دقت و خطا همیشه هم‌جهت نیستند:** در چند آزمایش خطای اعتبارسنجی بدتر ولی دقت بهتر شد؛ هر دو را باید گزارش کرد.
- **اعتبار آماری:** با ۶۷۹ نمونه، تفاضل‌های زیر ۱ واحد درصد در محدوده ی نوفه‌اند؛ چند-seed و بازه ی اطمینان لازم است.
- **پیش‌پردازش باید ثابت بماند:** Letterbox یا کشیدگی مستقیم، هر کدام که باشد، باید در آموزش و استنتاج یکسان باشد.
- **اطمینان Softmax صحت نیست:** پیش‌بینی‌های غلط با اطمینان ۹۷–۱۰۰٪ وجود دارند؛ تشخیص OOD به سازوکار جداگانه نیاز دارد.

۷.۴ محدودیت‌ها و ناسازگاری‌های گزارش منبع

جدول ۸۵ — فهرست ناسازگاری‌ها و نحوه ی برخورد با آن‌ها

مورد	توضیح	برخورد در این گزارش
آزمایش ۰۴	گزارش طبقه‌بندی با ۷۸۷ نمونه و دقت ۷۹.۰۳٪ در تضاد با لاگ (۷۲.۴۵٪) و ۶۷۹ نمونه	لاگ آموزش و Result.json مبنا؛ جدول‌ها با هشدار آمدند

مورد	توضیح	برخورد در این گزارش
آزمایش ۰۶	ماتریس درهم‌ریختگی برابر ماتریس weight_decay = ۰ (جمع قطر ۵۱۷ ≠ ۷۸.۶۵٪)	هشدار؛ تحلیل بر مبنای گزارش طبقه‌بندی
آزمایش ۰۸	دقت BCE: ۷۸.۷۹٪ (متن) در برابر ۷۸.۴۹٪ (Result.json)؛ CE جداگانه در Result.json نیست (۸۰.۴۱٪)	هر دو عدد ذکر شد
آزمایش ۰۵	ادعای «در همه ی اپوک‌ها بهتر» دقیق نیست (اپوک ۳ و ۶)	تصحیح شد (۸ از ۱۰ و ۹ از ۱۰)
آزمایش ۰۷	اندازه ی مجموعه ی آموزش ۲,۷۱۲ در برابر ۲,۷۱۳ در جاهای دیگر؛ پشتیبانی کلاس kamyun برابر ۱۰۰ (به‌جای ۹۹)	به تفاوت جزئی تقسیم نسبت داده شد
آزمایش ۰۹	اپوک‌های میانی لاگ ثبت نشده‌اند	همان جدول منبع آورده شد
آزمایش ۱۱	جمله ی پایانی منبع ناقص است	مفهوم آن در متن بازنویسی شد
بخش ۵.۷	پیش‌پردازش Letterbox در استنتاج در برابر Checkpoint بدون Letterbox	به‌عنوان نگرانی جدی مطرح شد

۸. معرفی گزارش نهایی و ادامه ی مطالعه

این گزارش مسیر **آزمایش‌ها و تکامل مدل** را تا انتخاب مدل ResNet۱۸ و بررسی OOD روایت کرد. گام آخر پروژه، یعنی **ارزیابی نهایی روی داده ی آزمون و سامانه ی تصمیم‌گیری OOD**، در فایل دوم با عنوان زیر مستندسازی شده است:

فایل بعدی: «گزارش نهایی پیش‌بینی و تشخیص OOD» — فایل `FinalPredict_OOD_Report_FA.docx` (مستند نوت‌بوک `FinalPredict_OOD.ipynb`)

فایل نهایی شامل چه چیزهایی است؟ خلاصه ی محتوای آن، به‌همراه ارتباطش با بخش‌های همین گزارش:

جدول ۸۶ — محتوای گزارش نهایی و ارتباط آن با این گزارش

موضوع در گزارش نهایی	محتوا	ارتباط با این گزارش
مقدمه و بیان مسئله	طبقه‌بندی ۸ کلاسه + رد کردن کلاس ناشناخته ی Neysan/Nissan؛ طراحی چهار وضعیت known / needs_review / unknown / known_failure	بخش ۶
تعریف مدل و پیش‌پردازش	Checkpoint نهایی ResNet۱۸، ایمن‌سازی مدل ۸ خروجی (بدون نورون Neysan)، ابعاد ۲۲۴ و نرمال‌سازی ImageNet	بخش‌های ۲.۲، ۲.۳، ۵.۷
پروتکل داده	مجموعه‌های test۰ تا test۴ و پوشه ی مستقل dataset/neysan؛ محدودیت برچسب واقعی در داده ی ترکیبی و GROUND_TRUTH_CSV	بخش ۶.۳
بانک ویژگی Mahalanobis	میانگین هر کلاس، کوواریانس Ledoit–Wolf، فاصله ی کمینه به‌عنوان امتیاز	بخش ۶.۲
امتیاز Energy و کالیبراسیون آستانه	فرمول انرژی با T = ۱؛ آستانه‌ها با صدک ۹۵ روی اعتبارسنجی داده ی شناخته‌شده؛ قاعده ی تصمیم سلسله‌مراتبی	بخش‌های ۶.۲ و ۶.۴
مفهوم known_failure	تعریف دقیق شکست آشکارساز برای Neysan واقعی	بخش ۶.۴
باگ خلاصه ی نهایی	فیلد Neysan Known Failures از DataFrame اشتباه محاسبه می‌شود؛ مقدار ۰.۰۰۰۰۰۰۰ معنایی ندارد؛ راه‌حل و فرمول‌های صحیح نرخ شناسایی و نرخ جاافتادگی	— (فقط در گزارش دوم)
ارزیابی کلاس‌های شناخته‌شده	خطا، دقت، ماتریس درهم‌ریختگی خام و نرمال‌شده، گزارش طبقه‌بندی، wrong_predictions.csv	بخش‌های ۲.۷ و ۳
تحلیل Grad-CAM	گروه‌های پیش‌بینی غلط، needs_review، unknown و Neysan شناسایی‌شده/جاافتاده	بخش ۵.۲
خروجی‌ها و تکرارپذیری	فهرست فایل‌های ۷ = seed، CSV، صدک ۰.۹۵، آستانه ی اطمینان ۰.۷۰، ترتیب قطعی بارگذار	بخش ۲.۴

موضوع در گزارش نهایی	محتوا	ارتباط با این گزارش
محدودیت‌ها و نتیجه‌گیری	OOD بودن بدون کلاس Neysan، آستانه ی صدکی، نیاز به ground truth، ایده ی دو مرحله‌ای	بخش‌های ۶ و ۷

راهنمای خواننده: برای ادامه ی مطالعه به **گزارش نهایی (فایل دوم)** مراجعه کنید: در آن اشتقاق ریاضی Energy و Mahalanobis، قاعده ی تصمیم چهارگانه، توضیح کامل باگ خلاصه ی نهایی و فرمول‌های صحیح نرخ شناسایی/جاافتادگی Neysan آمده است. به‌ویژه بخش «مشکل تجمیع در خلاصه ی نهایی» را بخوانید؛ زیرا عدد Neysan Known Failures = ۰ در آن فایل **شکست‌نخوردن آشکارساز را نشان نمی‌دهد**.

پیوست الف: واژه‌نامه

جدول ۸۷ — واژه‌نامه ی اصطلاحات فنی

واژه	معادل فارسی / توضیح
Overfitting	بیش‌برازش؛ حفظ داده ی آموزش و تعمیم ضعیف
Underfitting	کم‌برازش؛ ظرفیت یا آموزش ناکافی
Epoch	اپوک؛ یک گذر کامل روی داده ی آموزش
Batch	دسته؛ زیرمجموعه ی نمونه‌ها در یک گام بهینه‌سازی
Augmentation	افزایش داده؛ تولید نسخه‌های تغییریافته ی تصادفی
Dropout	حذف تصادفی نورون‌ها هنگام آموزش
Pooling / GAP	ادغام / ادغام میانگین سراسری
Weight Decay	کاهش وزن (منظم‌سازی L۲)
Learning Rate Scheduler	زمان‌بند نرخ یادگیری
Transfer Learning / Fine-Tuning	یادگیری انتقالی / تنظیم دقیق
Feature Extraction	استخراج ویژگی با backbone ثابت
Letterbox	تغییر اندازه با حفظ نسبت ابعاد و پر کردن حاشیه
Confusion Matrix	ماتریس درهم‌ریختگی
Precision / Recall / F۱	دقت مثبت / بازیابی / میانگین هماهنگ
OOD	خارج از توزیع (Out-of-Distribution)
Energy Score / Mahalanobis	امتیاز انرژی / فاصله ی ماهالانوبیس
Grad-CAM	نقشه ی گرمایی وابسته به گرادیان برای توجیه تصمیم شبکه
Early Stopping	توقف زودهنگام آموزش
Checkpoint	نقطه ی ذخیره ی مدل (بهترین وزن‌ها)